Windows CMD Mastery

From Zero to Expert Windows Administration, Automation, Internals & Defensive Security Research

A Hands-On Command Prompt, Batch Scripting, Windows Administration and Security Research Field Guide

Volume 1

Parts I-V: Fundamentals, Command Reference, Parser Internals, Edge Cases, Batch Scripting
Labs 1-25, 41-65

Copyright and Disclaimer

Copyright (c) Windows CMD Mastery. All rights reserved. This manuscript is educational material. Every command is provided for use on systems you own or are explicitly authorized to administer. Run every lab inside an isolated virtual machine with a snapshot taken first. Commands that change system configuration, security settings, disks, boot data or encryption state are marked DANGER and can cause data loss. The author and publisher accept no liability for damage resulting from use of this material. Windows behavior differs between versions, editions and builds; verify each command against current Microsoft Learn documentation for your exact build. No malware, credential theft, evasion or unauthorized-access content is included; security chapters use benign simulations only.

How to Use This Book

Read in order the first time. Type every example; do not paste. Every lab lists its own objective, prerequisites, expected result and cleanup. Work inside a disposable VM.

Legend	Meaning
[B] Beginner	No prior knowledge assumed
[I] Intermediate	Needs the earlier chapters
[A] Advanced	Needs parser and batch fundamentals
[X] Expert	Internals, edge cases, design trade-offs

Callout	Meaning
SAFETY (green)	How to keep a lab contained and reversible
DANGER / CAUTION (red)	Can destroy data, change boot or security state, or needs a snapshot
SECURITY (orange)	Defensive implication, abuse potential, detection note
NOTE (blue)	Clarification or version note

Table of Contents

Copyright and Disclaimer 2

How to Use This Book 2

Table of Contents 3

Part I - Windows Command Line Fundamentals 7

Chapter 1. What Windows, a Shell and CMD.exe Actually Are 9

1.1 The operating system in one paragraph . . . 9
1.2 What is a command-line interface? . . . 9
1.3 Shell, console, terminal: four words people mix up . . . 9
1.4 What CMD.exe is . . . 9
1.5 CMD versus PowerShell versus Windows Terminal . . . 10
1.6 Interactive mode versus batch mode . . . 10
1.7 Opening CMD . . . 10
1.8 Your first check: which prompt am I in? . . . 11

Chapter 2. Anatomy of the Command Prompt 12

2.1 Reading the prompt . . . 12
2.2 Drives are per-process, and each drive remembers its own folder . . . 12
2.3 The five things every command line has . . . 12
2.4 Streams, made visible . . . 13
2.5 Exit status . . . 13
2.6 Environment variables, first look . . . 13

Chapter 3. Your First Commands 14

3.1 dir - list a folder . . . 14
3.2 cd - change directory . . . 14
3.3 cls, echo, type, more, help, tree . . . 15
3.4 A guided first session (Lab 1 preview) . . . 15
3.5 Getting unstuck . . . 15
3.6 Command history and editing keys . . . 16

Part II - CMD Command Reference 17

Chapter 4. How to Read a Command Card 19

Chapter 5. Navigation and Display Commands 20

Chapter 6. File and Folder Commands 25

Chapter 7. Variables, Flow and Process Commands 30

Chapter 8. Text Filtering and Comparison Commands 34

Chapter 9. System Information Built-ins 37

Part III - CMD Parser Internals 39

Chapter 10. The Phases of Command Interpretation 41

10.1 Why this chapter matters . . . 41
10.2 The phases at a glance . . . 41
10.3 Phase 1: percent expansion . . . 41
10.4 Phase 2: special characters and tokenizing . . . 42
10.5 The block-parse rule and the classic surprise . . . 43
10.6 Phase 3: echo . . . 44
10.7 Phase 4: FOR variables . . . 44
10.8 Phase 5: delayed expansion . . . 44
10.9 Phase 6: redirection . . . 44
10.10 Phase 7: CALL and the second pass . . . 44
10.11 Phase 8: execution . . . 44
10.12 A worked trace . . . 45

Chapter 11. Operators, Redirection, Pipes and Nested Commands 47

11.1 Command separators . . . 47
11.2 The three standard streams . . . 47
11.3 Redirection operators . . . 47
11.4 Pipes . . . 48
11.5 cmd /c and cmd /k . . . 48
11.6 Nesting and precedence . . . 49
11.7 Interactive versus batch differences . . . 49

Chapter 12. Command Lookup, PATH, PATHEXT and Argument Parsing 52

12.1 What happens when you type a name . . . 52
12.2 Two parsers, two sets of rules . . . 53
12.3 Environment inheritance . . . 53

Part IV - CMD Edge Cases 55

Chapter 13. Expert Edge-Case Catalog 57

13.1 Spaces in paths . . . 57
13.2 Quotes inside quotes . . . 57
13.3 Trailing spaces . . . 57
13.4 Special characters as data . . . 57
13.5 Parentheses . . . 58
13.6 Redirection and pipe quirks . . . 58
13.7 FOR variable details . . . 58
13.8 CALL expansion . . . 58
13.9 Code pages and encodings . . . 59
13.10 Unicode limitations . . . 59
13.11 Line endings (CRLF) . . . 59
13.12 Standard input behavior . . . 59
13.13 Interactive versus batch: expansion-timing summary . . . 59

Part V - Batch Scripting 64

Chapter 14. Your First Scripts: Variables and Arguments 66

14.1 What a batch file is . . . 66
14.2 A professional script header . . . 66
14.3 Variables . . . 66
14.4 Environment variables and dynamic variables . . . 67
14.5 Command-line arguments . . . 67
14.6 Validating input . . . 67

Chapter 15. Control Flow: IF, GOTO, FOR and FOR /F 69

15.1 IF . . . 69
15.2 GOTO and labels . . . 69
15.3 FOR (files and sets) . . . 69
15.4 FOR /F: parsing text . . . 70

15.5 Nested loops and counters . . . 70
15.6 Arrays and pseudo-data structures . . . 71

Chapter 16. Subroutines, Errors, Logging and Testing 72

16.1 Subroutines with CALL . . . 72
16.2 ERRORLEVEL and exit codes . . . 72
16.3 A reusable logging function . . . 73
16.4 Privilege and prerequisite checks . . . 73
16.5 Temporary files and locking . . . 73
16.6 Retry and timeout . . . 74
16.7 Debugging . . . 74
16.8 Testing batch scripts . . . 74
16.9 Performance and portability . . . 75
16.10 When batch is the wrong tool . . . 75

Part XXIV - Hands-On Labs (Beginner Set) 80

Chapter 17. Beginner Labs 1-25 82

Back Matter 93

Chapter 18. Glossary 95

Chapter 19. Command Index (Volume 1) 96

Chapter 20. References and Verification Notes 97

Chapter 21. What Comes Next 98

Part I - Windows Command Line Fundamentals

Starting from absolute zero: what a shell is, what CMD.exe does, and how to read a prompt.

Chapter 1. What Windows, a Shell and CMD.exe Actually Are

1.1 The operating system in one paragraph

An **operating system** (OS) is the software that sits between your programs and your hardware. Programs never touch the disk, the network card or memory directly. They ask the OS, and the OS decides whether to allow it. Windows is Microsoft's OS. Its core, the **kernel**, runs in a privileged CPU mode called **kernel mode**. Everything you normally run (a browser, Notepad, CMD) runs in **user mode**, where it must politely request services from the kernel through documented interfaces (the Windows API, or **Win32 API**).

Analogy: think of the kernel as a hotel's front desk. Guests (programs) cannot walk into the storeroom (disk) or use the master key (hardware). They ask the front desk, which checks who they are and what they are allowed to do.

1.2 What is a command-line interface?

A **command-line interface** (CLI) lets you control a computer by typing text commands instead of clicking. You type a line, press Enter, and the computer runs it and prints text back. A **graphical user interface** (GUI) does the same jobs with windows and buttons.

Why bother? Text commands can be **repeated exactly**, **saved in a file** (a script), **run on a hundred machines**, **run without anyone logged in**, and **recorded in logs**. A mouse click cannot easily do any of those.

1.3 Shell, console, terminal: four words people mix up

Term	Meaning	Windows example
Shell	The program that reads your typed line, interprets it and starts other programs	`cmd.exe`, `powershell.exe`
Console window	The visible window that shows text and receives keystrokes	The classic black window
Console host	The system component that owns the console, draws text and passes keys to the shell	`conhost.exe`
Terminal application	A modern app that hosts one or more shells in tabs	Windows Terminal (`wt.exe`)

The important separation: **CMD is the shell (the interpreter). Windows Terminal is only a window that displays a shell.** You can run CMD inside Windows Terminal, inside the old console window, or with no window at all (a service or scheduled task launching `cmd.exe /c`).

> **NOTE.** Windows 11 (version 22H2 and later) uses Windows Terminal as the default terminal application on new installs. On Windows 10 the classic console host is still the default unless you change it in Settings. Behavior of the CMD interpreter itself is the same in both.

1.4 What CMD.exe is

`cmd.exe` is the Windows **command interpreter**. It lives at `C:\Windows\System32\cmd.exe` (a 64-bit build; a 32-bit copy is in `C:\Windows\SysWOW64`). It does three jobs:

1. **Reads** a line of text (from the keyboard, or from a script file).
2. **Parses** it: decides which parts are commands, arguments, operators and variables (Part III covers this in depth).
3. **Executes** it: either runs a **built-in command** that lives inside `cmd.exe` itself (like `cd` and `echo`), or asks Windows to start an **external program** (like `ping.exe`).

Its ancestry is `COMMAND.COM` from MS-DOS. That is why CMD has odd rules (`%` variables, drive letters, `goto` labels) that look old. They are old, and they are frozen for compatibility. Microsoft does not add major new features to CMD; new automation work goes into PowerShell.

1.5 CMD versus PowerShell versus Windows Terminal

Aspect	CMD	PowerShell	Windows Terminal
What it is	Text-based shell	Object-based shell and scripting language	Window/tab host, not a shell
Pipeline carries	Plain text	.NET objects	n/a
Script type	`.bat`, `.cmd`	`.ps1`	none
Best for	Tiny glue scripts, legacy compatibility, recovery environments	Modern administration, APIs, JSON	Running either comfortably
Present in WinRE/Setup	Yes	Usually no	No

CMD is still worth mastering because it is present on every Windows install, works in recovery media (WinPE/WinRE), is used by installers and legacy tooling, and because attackers and defenders both see it constantly in process telemetry. Part XXVIII gives an honest capability comparison.

1.6 Interactive mode versus batch mode

- **Interactive**: you type one line at a time and read the result.
- **Batch**: CMD reads lines from a file (`.bat` or `.cmd`) and runs them without asking you.

The same interpreter runs both, but a few behaviors differ (for example, how `%` is treated for `FOR` variables, and how `set /p` and `pause` wait for input). These differences are catalogued in Chapter 11.

1.7 Opening CMD

Any of these works:

- Press **Win+R**, type `cmd`, press Enter.
- Press the Start key, type `cmd`, press Enter.
- In File Explorer, click the address bar, type `cmd`, press Enter (opens CMD in that folder).
- Right-click Start -> **Terminal** (Windows 11) and choose a Command Prompt tab.

Standard versus Administrator CMD

Windows has **User Account Control (UAC)**. Even if your account is in the Administrators group, programs start with a **filtered token** (limited rights). Only when you explicitly choose **Run as administrator** does

Windows create a second, **elevated** token with full administrative rights.

	Standard CMD	Elevated (Administrator) CMD
Title bar	`Command Prompt`	`Administrator: Command Prompt`
Can edit files under `C:\Windows`?	No	Yes
Can change services, firewall, boot data?	Mostly no	Yes
Risk of an accidental typo	Limited	High

> **SAFETY.** Do daily learning in a **standard** prompt. Open an elevated prompt only for the specific labs that say "Administrator required," and close it afterward.

1.8 Your first check: which prompt am I in?

```
whoami
whoami /groups | find "Mandatory Label"
```

Expected on a standard prompt: your account name, and a line ending in `Medium Mandatory Level`. On an elevated prompt the label reads `High Mandatory Level`. The integrity-level idea is explained fully in Part X.

Chapter 2. Anatomy of the Command Prompt

2.1 Reading the prompt

```
C:\Users\Alice>
```

Piece	Meaning
`C:`	The current **drive** (a volume mounted with a letter)
`\` after `C:`	The **root** of that drive
`Users\Alice`	The **path** of the current directory (folder)
`>`	The **prompt terminator**: "I am ready for input"
Everything you type after `>`	The **command line**

The **current directory** (also called the **working directory**) is the folder CMD treats as "here." If you type a filename without a full path, CMD looks in the current directory first for relative paths. Each process has its own current directory, and a child process inherits its parent's.

Analogy: the current directory is the room you are standing in. `notes.txt` means "the notes.txt in this room." `C:\Work\notes.txt` is a complete street address that works from anywhere.

2.2 Drives are per-process, and each drive remembers its own folder

CMD keeps a current directory for every drive letter. This surprises beginners:

```
C:\Users\Alice> cd /d D:\Projects
D:\Projects> C:
C:\Users\Alice>
```

Typing `D:` alone switches drive and returns you to the folder D: last used. Plain `cd D:\Other` on a different drive changes D:'s remembered folder but does **not** switch drives. The `/d` switch does both.

2.3 The five things every command line has

```
program   arguments...   redirection   exit code
```

1. **Input**: keyboard (standard input, `stdin`), a file, or a pipe.
2. **Output**: normal results go to **standard output** (`stdout`).
3. **Error output**: error messages go to **standard error** (`stderr`). It is a separate stream even though both usually show on screen.
4. **Environment**: named text values (variables) every process inherits, such as `PATH`.
5. **Exit status**: a number a finished program hands back. By convention `0` means success and nonzero means some kind of failure. CMD exposes it as `ERRORLEVEL`.

Diagram of one command:

```
          keyboard / file / pipe
                  |
               [stdin]
                  v
   +-----------------------------+
   |         a program           |--> [stdout] --> screen / file / pipe
   |  (env vars, current dir)    |--> [stderr] --> screen / file
   +-----------------------------+
                  |
             exit code (ERRORLEVEL)
```

2.4 Streams, made visible

Run these two and watch where the text goes:

```
echo hello
dir C:\this_folder_does_not_exist
```

The first prints `hello` (stdout). The second prints `File Not Found` or `The system cannot find the path specified.` (stderr). Now separate them:

```
dir C:\this_folder_does_not_exist > out.txt 2> err.txt
type out.txt
type err.txt
```

`out.txt` will be empty or nearly so and `err.txt` holds the message. Proof that they are different streams. Cleanup: `del out.txt err.txt`.

2.5 Exit status

```
ping -n 1 127.0.0.1 >nul
echo %ERRORLEVEL%
ping -n 1 no-such-host.invalid >nul
echo %ERRORLEVEL%
```

The first is `0`. The second is nonzero (commonly `1`). `%ERRORLEVEL%` reads the exit code of the most recent command. Be careful: any later command overwrites it. Save it immediately if you need it (`set RC=%ERRORLEVEL%`).

> **NOTE.** Success is not always `0` for every Windows tool. `robocopy` returns bit-flag values 0-7 for success-with-details and 8 or higher for failure. `findstr` and `find` return 1 for "no match," which is not an error. Part XXVII documents these.

2.6 Environment variables, first look

```
echo %USERNAME%
echo %COMPUTERNAME%
echo %CD%
set USER
```

`%NAME%` asks CMD to substitute the variable's value **before** running the command. `set USER` with no = lists every variable whose name starts with `USER`. The next parts explain exactly when substitution happens.

Chapter 3. Your First Commands

For each command below: what it does, the tiny example, the expected output, and one experiment. Full 22-field reference cards for many of these appear in Part II.

3.1 dir - list a folder

```
dir
```

Sample output:

```
 Volume in drive C has no label.
 Volume Serial Number is 1A2B-3C4D

 Directory of C:\Users\Alice

09/30/2026  10:14 AM    <DIR>          .
09/30/2026  10:14 AM    <DIR>          ..
09/29/2026  08:02 PM    <DIR>          Documents
09/29/2026  08:02 PM    <DIR>          Downloads
09/30/2026  09:55 AM             1,204 notes.txt
               1 File(s)          1,204 bytes
               4 Dir(s)  212,345,675,776 bytes free
```

Read it: `.` means "this folder," `..` means "the parent folder." `<DIR>` marks folders. Dates follow your regional format. The last lines are totals and free space.

Experiments:

```
dir /a          & rem include hidden and system items
dir /b          & rem bare names only, one per line (great for scripts)
dir /s /b *.txt & rem search this folder and all subfolders for .txt files
dir /o:-s       & rem sort by size, largest first
dir /a:d /b     & rem folders only
```

3.2 cd - change directory

```
cd Documents
cd ..
cd \
cd /d D:\
cd
```

Form	Effect
`cd Documents`	Move into a subfolder (relative path)
`cd ..`	Move up one level
`cd ..\..`	Move up two levels
`cd \`	Jump to the root of the current drive
`cd /d D:\Data`	Change drive and folder together
`cd` (no argument)	**Print** the current directory

If a path has spaces, quote it: `cd "C:\Program Files"`. (CMD's `cd` is lenient about unquoted spaces, but relying on that is a habit that breaks other commands.)

3.3 cls, echo, type, more, help, tree

```
cls
echo Hello, world
echo Line one> demo.txt
echo Line two>> demo.txt
type demo.txt
type demo.txt | more
help dir
dir /?
tree /f C:\Users\Alice\Documents
```

- `cls` clears the screen. It does not delete anything.
- `echo text` prints text. `echo.` (with a dot) or `echo(` prints a blank line reliably. `echo` alone prints the ECHO state ("ECHO is on.").
- `>` creates or **overwrites** a file with the output. `>>` **appends**. Beware: `echo Line one> demo.txt` writes "Line one" with no trailing space only because the `>` has no space before it. With a space you get a trailing space in the file.
- `type file` prints a file's contents. `more` pauses output page by page (Space = next page, Q = quit).
- `help` alone lists built-in commands with one-line descriptions. `help <cmd>` and `<cmd> /?` show syntax. For external tools use `<tool> /?`.
- `tree` draws a folder hierarchy. `/f` includes files. `/a` uses plain ASCII lines.

Cleanup: `del demo.txt`

3.4 A guided first session (Lab 1 preview)

```
cd /d %USERPROFILE%
mkdir cmdlab
cd cmdlab
echo alpha> a.txt
echo beta> b.txt
dir /b
type a.txt
copy a.txt c.txt
ren c.txt gamma.txt
dir /b
cd ..
rmdir /s /q cmdlab
```

Each line: go to your home folder, make a practice folder, enter it, make two files, list them by name, view one, copy it, rename the copy, list again, back out, remove the whole practice folder (`/s` = include contents, `/q` = no confirmation).

> **DANGER / CAUTION.** `rmdir /s /q` and `del /s /q` delete without confirmation and without the Recycle Bin. Always `echo` or `dir` the exact path first, and never run them from a folder you have not verified with `cd`.

3.5 Getting unstuck

Situation	What to do
A command seems to hang	Press **Ctrl+C** to interrupt it
Output scrolls off screen	Add `| more`, or redirect to a file and open it
You do not know a command's options	`command /?`
You typed the wrong thing	Up arrow recalls history; **F7** shows a history popup
Text looks garbled	Likely a code page issue; see Chapter 13
Window title says Administrator and you did not mean to	Close it, reopen normally

3.6 Command history and editing keys

Key	Action
Up / Down	Previous / next command in history
F1	Copy one character from the previous command
F3	Repeat the previous command
F7	Popup list of history
F8	Cycle history entries that start with what you typed
Tab	Complete file and folder names (Shift+Tab goes backward)
Ctrl+C	Interrupt the running command
Ctrl+Z then Enter	End-of-input marker when typing to stdin (for example `copy con file.txt`)

`doskey /history` prints the command history. `doskey` can also define macros; see the reference card in Part II.

Part II - CMD Command Reference

Volume 1 covers the core built-ins and everyday utilities. Networking, services, registry, tasks, logs, recovery and the security-relevant utilities get their full cards in Volumes 2 and 3 next to the chapters that explain them.

Chapter 4. How to Read a Command Card

Every card uses the same fields. Where a field does not apply it says so instead of being dropped.

Field	What it tells you
Category	Built-in (lives inside `cmd.exe`) or External (a separate `.exe`)
Purpose / Why	What it does and the reason it exists
Syntax, Switches	Exact forms; only the switches worth knowing
Beginner, Intermediate, Advanced, Real-world	Four graded examples
Output	What you should see and how to interpret it
Exit / Errors	Return codes and typical failure messages
Admin, Versions	Elevation needs and Windows 10/11 notes
Mistakes, Security	Common traps and defensive implications
Lab, Cleanup	A safe experiment and how to undo it
Related, PowerShell	Neighbor commands and the modern equivalent

NOTE. Built-in versus external matters. Built-ins (`cd`, `set`, `echo`, `dir`, `copy`, `type` ...) run inside `cmd.exe`, so `help` documents them and they cannot be replaced by dropping a file on the PATH. External tools (`ping.exe`, `find.exe`, `robocopy.exe`) are files; CMD searches for them, which is where the path-resolution risks of Part XXI come from.

SAFETY. Every card's lab runs inside a scratch folder. Create it once and reuse it: `mkdir "%TEMP%\cmdlab" & cd /d "%TEMP%\cmdlab"`. Cleanup for the whole book's file labs is `cd /d "%TEMP%" & rmdir /s /q cmdlab`.

Chapter 5. Navigation and Display Commands

DIR [Built-in, file system]

Field	Detail
Purpose / Why	Lists files and folders with attributes, dates, sizes. Exists because you cannot manage what you cannot see.
Syntax	`dir [path] [/a[:attrs]] [/b] [/s] [/o[:order]] [/t[:timefield]] [/q] [/x] [/r]`
Key switches	`/a:h` hidden, `/a:d` dirs, `/a:-d` files only, `/b` bare, `/s` recurse, `/o:n` name, `/o:s` size, `/o:d` date, `/t:c` creation time, `/t:a` last access, `/q` show owner, `/x` show 8.3 short names, `/r` show alternate data streams
Beginner	`dir`
Intermediate	`dir /a:-d /o:-s C:\Windows`
Advanced	`dir /s /b /a:-d "%ProgramFiles%" 2>nul | find /c /v ""` (count files under a tree)
Real-world	`dir /t:w /o:-d C:\Logs` to find the newest log by last-write time
Output	Header (volume, serial), listing, file count and byte total, free space. Sizes use your locale's thousands separator.
Exit / Errors	0 on success; 1 with `File Not Found` if nothing matches; `The system cannot find the path specified.` for bad paths
Admin	No, but protected folders may deny listing (`Access is denied.`)
Versions	`/r` (streams) needs NTFS and Vista or later. Behavior identical on 10 and 11.
Mistakes	Parsing `dir` text in scripts (locale-dependent). Use `dir /b` or `for` instead. Forgetting `/a` hides system/hidden files.
Security	`dir /q` reveals file owners; `dir /r` reveals hidden data in alternate streams (Part VII).
Lab / Cleanup	`echo x> a.txt & attrib +h a.txt & dir & dir /a:h` then `attrib -h a.txt & del a.txt`
Related	`tree`, `where`, `for /r`
PowerShell	`Get-ChildItem -Force`, alias `gci`/`ls` (returns objects, not text)

CD / CHDIR [Built-in, navigation]

Field	Detail
Purpose / Why	Changes or prints the current directory. Everything relative depends on it.
Syntax	`cd [/d] [drive:][path]`, `cd ..`, `cd \`, `cd`
Key switches	`/d` also switches the current drive
Beginner	`cd Documents`, then `cd ..`

Field	Detail
Intermediate	`cd /d "D:\My Projects"`
Advanced	`cd /d "%~dp0"` inside a batch file (go to the script's own folder)
Real-world	Start every deployment script with `cd /d "%~dp0"` so relative paths behave no matter where it was launched from
Output	No output on success; bare `cd` prints the current directory
Exit / Errors	1 with `The system cannot find the path specified.` if missing. `The filename, directory name, or volume label syntax is incorrect.` for illegal characters.
Admin	No
Versions	`cd` accepts UNC paths only if command extensions allow; CMD will warn `UNC paths are not supported` and stay put. Use `pushd \\server\share` to map a temporary drive letter.
Mistakes	Forgetting `/d` when changing drives; forgetting that `cd` does not change the drive
Security	Scripts that rely on the caller's current directory can execute files from a hostile folder (Part XXI)
Lab / Cleanup	`cd /d %TEMP% & cd & cd ..\.. & cd` (no cleanup needed)
Related	`pushd`, `popd`, `%CD%`, `%~dp0`
PowerShell	`Set-Location` (alias `cd`), `Push-Location`

CLS, TITLE, COLOR, PROMPT [Built-ins, display]

Field	Detail
Purpose / Why	Cosmetic control of the console: clear screen, window title, colors, prompt text
Syntax	`cls`; `title text`; `color [bf]`; `prompt [text]`
Key switches	`color 0A` = black background, light-green text; `color` alone resets. `prompt $P$G` = path and `>`; `$D` date, `$T` time, `$_` newline
Beginner	`title Practice Window`, `color 0A`, `cls`
Intermediate	`prompt $P$_$G` gives a two-line prompt
Advanced	`prompt $E[32m$P$E[0m$G` uses ANSI escape codes (works in Windows Terminal and modern conhost with virtual terminal processing)
Real-world	Set a red title `Administrator - PRODUCTION` when working on servers to avoid mistakes
Output	Immediate visual change
Exit / Errors	`color` returns 1 if foreground and background are the same
Admin	No
Versions	ANSI/VT sequences are honored on Windows 10 1511 and later when the console enables VT mode
Mistakes	Changing `prompt` in a script and forgetting it only affects that CMD session
Security	Look-alike prompts can disguise elevated sessions in training demos; the reliable check is the window title and `whoami /groups`
Lab / Cleanup	`prompt $P$G` restores the default
Related	`echo`, `mode`, `doskey`

Field	Detail
PowerShell	`$Host.UI.RawUI.WindowTitle`, `function prompt {}`

ECHO [Built-in, output]

Field	Detail
Purpose / Why	Prints text, and turns command display on or off in scripts
Syntax	`echo [text]`, `echo on`, `echo off`, `@echo off`
Key switches	`@` before a command suppresses that one line's echo; `echo.` and `echo(` print a blank line
Beginner	`echo Hello`
Intermediate	`echo Total: %COUNT% files> report.txt`
Advanced	`echo(%VALUE%` prints a value even if it is empty or begins with `on`/`off`
Real-world	Progress and logging lines in batch files
Output	The text, after variable expansion
Exit / Errors	Always 0
Admin	No
Versions	Same on all versions
Mistakes	`echo %VAR%` when `VAR` is empty prints `ECHO is on.`; special characters (`& | < > ^ %`) need escaping (Chapter 10)
Security	Echoing secrets into logs or console history exposes them (Part XIX)
Lab / Cleanup	`echo(` then `echo.` then `echo` and compare
Related	`type`, `set /p`, `more`
PowerShell	`Write-Output`, `Write-Host`

TYPE and MORE [Built-in and External, viewing files]

Field	Detail
Purpose / Why	`type` prints file contents to stdout; `more` pages long output
Syntax	`type file...`; `more [/e] [/c] [/p] [+n] [file]`
Key switches	`more /c` clears screen before each page; `more +n` starts at line n
Beginner	`type notes.txt`
Intermediate	`type a.txt b.txt > combined.txt`
Advanced	`type file.txt | findstr /i "error"`
Real-world	Quick log tail (`type log.txt | more`) on machines lacking other tools
Output	Raw text. Binary files print garbage and may beep.
Exit / Errors	1 with `The system cannot find the file specified.`

Field	Detail
Admin	No (but file ACLs apply)
Versions	`type` can read alternate streams: `type file.txt:stream`
Mistakes	`type` on huge files floods the console; UTF-16 files show spaced letters or nulls
Security	`type` respects NTFS permissions; it is not a bypass
Lab / Cleanup	`dir /b C:\Windows\System32 > list.txt & more list.txt` then `del list.txt`
Related	`find`, `findstr`, `sort`, `clip`
PowerShell	`Get-Content` (alias `cat`, `type`)

TREE [External, display]

Field	Detail
Purpose / Why	Draws a directory tree
Syntax	`tree [path] [/f] [/a]`
Key switches	`/f` include files, `/a` ASCII characters (safe for logs and old code pages)
Beginner	`tree`
Intermediate	`tree /f /a "%USERPROFILE%\Documents" > tree.txt`
Advanced	Combine with `find` to locate a name inside the drawn tree
Real-world	Documenting a folder layout in a ticket
Output	Indented box-drawing lines; with `/a`, plus signs, pipes and backslashes
Exit / Errors	0 or nonzero if the path is invalid
Admin	No
Versions	Same on 10 and 11
Mistakes	Redirecting box-drawing output to a file and opening it with a different code page shows garbage; use `/a`
Security	Output reveals directory names you may not want in a shared ticket
Lab / Cleanup	`tree /a C:\Windows\Logs`
Related	`dir /s`, `robocopy /l`
PowerShell	`Get-ChildItem -Recurse`, or `tree` also works from PowerShell

HELP and /? [Built-in, documentation]

Field	Detail
Purpose / Why	On-box documentation for CMD built-ins and most tools
Syntax	`help [command]`, `command /?`
Beginner	`help copy`

Field	Detail
Intermediate	`help for > forhelp.txt & more forhelp.txt`
Advanced	`help set` covers `%VAR:~n,m%` and `%VAR:a=b%` syntax; read it once fully
Real-world	Verifying exact switch names before scripting
Output	Usage text
Exit / Errors	`This command is not supported by the help utility` for external tools; use `tool /?`
Admin	No
Versions	Text varies slightly by build; `help` lists only built-ins
Mistakes	Assuming `help` documents `ipconfig` or `netsh`; use `ipconfig /?`
Security	Trusting random web snippets instead of `/?` leads to dangerous switches; verify locally first
Lab / Cleanup	`help | find /c /v ""` counts lines; no cleanup
Related	Microsoft Learn Windows commands reference
PowerShell	`Get-Help`, `Get-Command`

Chapter 6. File and Folder Commands

MKDIR / MD and RMDIR / RD [Built-ins, folders]

Field	Detail
Purpose / Why	Create and remove directories
Syntax	`md path`, `rd [/s] [/q] path`
Key switches	`rd /s` removes the tree; `/q` suppresses the confirmation
Beginner	`md practice` then `rd practice`
Intermediate	`md a\b\c` (creates intermediates when extensions are on)
Advanced	`rd /s /q "%TEMP%\cmdlab" 2>nul` (silent idempotent cleanup)
Real-world	Deployment scripts creating dated backup folders
Output	None on success
Exit / Errors	`A subdirectory or file ... already exists.`; `The directory is not empty.`; `The process cannot access the file because it is being used by another process.`
Admin	Depends on the parent's ACL
Versions	Same
Mistakes	`rd /s /q` with an empty variable path (`rd /s /q %X%\`) can target the current drive root; always validate variables first
Security	Deleting folders that contain reparse points: `rd /s` removes the link, not the target, on modern Windows, but test in a lab first (Part VII)
Lab / Cleanup	`md t1\t2\t3 & tree /a t1 & rd /s /q t1`
Related	`del`, `pushd`
PowerShell	`New-Item -ItemType Directory`, `Remove-Item -Recurse`

COPY, XCOPY, MOVE [Built-in, Built-in and External, file transfer]

Field	Detail
Purpose / Why	Duplicate or relocate files. `xcopy` adds tree copying; `robocopy` (Part VII) is the modern replacement.
Syntax	`copy [/y] [/v] src dest`; `xcopy src dest [/s /e /i /y /d /h /k]`; `move [/y] src dest`
Key switches	`copy /y` no overwrite prompt, `/v` verify; `xcopy /e` include empty dirs, `/i` assume dest is a directory, `/h` hidden files, `/d` newer only
Beginner	`copy a.txt b.txt`
Intermediate	`copy a.txt+b.txt ab.txt` (concatenate text files)
Advanced	`xcopy "%~dp0data" "D:\backup\data" /e /i /y /h /d`

Field	Detail
Real-world	Small one-off folder copies where `robocopy` is overkill
Output	`1 file(s) copied.`
Exit / Errors	`copy`: 1 on failure. `xcopy`: 0 ok, 1 nothing found, 2 Ctrl+C, 4 init error, 5 disk write error
Admin	Only if the destination is protected
Versions	`xcopy` is deprecated in favor of `robocopy` but remains available
Mistakes	`copy` without `/y` inside scripts hangs waiting for an overwrite answer. Concatenating binary files without `/b` corrupts them.
Security	Copying preserves neither ownership nor full ACLs the way `robocopy /sec` does; forensic copies need proper tools
Lab / Cleanup	`echo 1> a.txt & copy a.txt b.txt & move b.txt c.txt & dir /b` then `del a.txt c.txt`
Related	`robocopy`, `replace`, `fc`
PowerShell	`Copy-Item`, `Move-Item`

DEL / ERASE and REN / RENAME [Built-ins, file removal and naming]

Field	Detail
Purpose / Why	Delete and rename files
Syntax	`del [/p] [/f] [/s] [/q] [/a:attrs] names`; `ren old new`
Key switches	`/p` prompt per file, `/f` force read-only, `/s` recurse, `/q` quiet, `/a:h` match hidden
Beginner	`del old.txt`, `ren a.txt b.txt`
Intermediate	`del /q *.tmp`
Advanced	`del /f /q /s "%TEMP%\cmdlab\*.*" 2>nul`
Real-world	Scheduled cleanup of aged temp files (paired with `forfiles`, Volume 3)
Output	Silent on success (`/s` lists deleted files)
Exit / Errors	`Could Not Find ...`; `Access is denied.`; `The process cannot access the file because it is being used by another process.`
Admin	If ACLs require it
Versions	`ren` cannot move across drives; use `move`
Mistakes	`del *` in the wrong folder; `del` deletes files in the given folder only, not subfolders, unless `/s`; wildcard `*.*` matches 8.3 aliases in odd cases
Security	`del` is not secure erase; contents can be recoverable; use `cipher /w` for free-space wiping (Part VII)
Lab / Cleanup	Run `del /p` on a test file and answer `N`
Related	`rd`, `attrib`, `cipher`
PowerShell	`Remove-Item`, `Rename-Item`

ATTRIB [External, file attributes]

Field	Detail
Purpose / Why	Show or change file attributes (read-only, hidden, system, archive)
Syntax	`attrib [+r|-r] [+a|-a] [+s|-s] [+h|-h] [path] [/s] [/d]`
Key switches	`/s` recurse into subfolders, `/d` include folders
Beginner	`attrib a.txt`
Intermediate	`attrib +r +h secret.txt`
Advanced	`attrib -h -s /s /d D:\*.*` to reveal items hidden by a mistaken command (review with `dir /a` first)
Real-world	Clearing read-only from a deployed tree before update
Output	One line per file: letters `A S H R` then path
Exit / Errors	0, or nonzero with `File not found`
Admin	Not for your own files; system files need ownership/rights
Versions	Same
Mistakes	Hiding files is not protection. Setting `+s +h` on a folder makes it invisible in Explorer defaults.
Security	Malware often sets Hidden and System on dropped files; defenders check `dir /a:hs` in suspicious folders
Lab / Cleanup	`echo x> t.txt & attrib +h +r t.txt & dir /b & dir /a:h /b & attrib -h -r t.txt & del t.txt`
Related	`icacls`, `dir /a`, `fsutil`
PowerShell	`Get-ItemProperty ... .Attributes`, `Set-ItemProperty`

MKLINK [Built-in, links]

Field	Detail
Purpose / Why	Creates symbolic links, directory junctions and hard links
Syntax	`mklink [/d] link target`; `mklink /j link targetDir`; `mklink /h link targetFile`
Key switches	`/d` directory symlink, `/j` junction, `/h` hard link
Beginner	`mklink /h copy.txt orig.txt`
Intermediate	`mklink /j "%TEMP%\cmdlab\jn" "%WINDIR%\Logs"`
Advanced	`dir /a:l` to list reparse points; `fsutil reparsepoint query jn`
Real-world	Redirect a large cache folder to another disk without changing the application
Output	`symbolic link created for link <<===>> target`
Exit / Errors	`You do not have sufficient privilege to perform this operation.` (symlinks need elevation, or Developer Mode on Windows 10 1703+)

Field	Detail
Admin	Symlinks yes unless Developer Mode; junctions and hard links no
Versions	Developer Mode removes the elevation requirement for symlinks on Windows 10 1703+ and Windows 11
Mistakes	Hard links cannot cross volumes and cannot point at folders; deleting a junction with `del` can behave unlike `rd` (use `rd` for junction folders)
Security	Link-following races are a real vulnerability class; defenders audit junctions in privileged writable locations
Lab / Cleanup	Create a junction to `%WINDIR%\Logs` in the scratch folder, `dir`, then `rd jn` (removes the link only)
Related	`fsutil`, `dir /al`, `icacls`
PowerShell	`New-Item -ItemType SymbolicLink/Junction/HardLink`

PUSHD and POPD [Built-ins, navigation stack]

Field	Detail
Purpose / Why	`pushd` remembers the current directory then changes; `popd` returns. Also maps UNC paths to temporary drive letters.
Syntax	`pushd path`, `popd`
Beginner	`pushd %TEMP%` ... `popd`
Intermediate	`pushd \\server\share` gives a drive letter automatically (Z: downward)
Advanced	Nested pushes make a stack; scripts should balance every `pushd` with a `popd`, including error paths
Real-world	Scripts that must visit several folders and return to where they started
Output	None; `pushd` with no argument may list the stored directories when command extensions are on
Exit / Errors	Same as `cd`
Admin	No
Versions	Same
Mistakes	Unbalanced pushes leave scripts in the wrong folder and leak mapped UNC drive letters
Security	Auto-mapped drives can persist under the session
Lab / Cleanup	`cd & pushd %WINDIR% & cd & popd & cd`
Related	`cd`, `net use`
PowerShell	`Push-Location`, `Pop-Location`

WHERE [External, command lookup]

Field	Detail
Purpose / Why	Finds files by name along the PATH or in a tree; the right way to see what a command name resolves to
Syntax	`where [/r dir] [/q] [/f] [/t] pattern`

Field	Detail
Key switches	`/r` recursive search from dir; `/q` quiet (exit code only); `/f` quote output; `/t` size and date
Beginner	`where ping`
Intermediate	`where notepad*`
Advanced	`where /r "%ProgramFiles%" *.dll 2>nul`
Real-world	Diagnosing which `python.exe` (or `git.exe`) a script will actually run
Output	Full paths in search order; the first is the one CMD runs. Current directory is searched first.
Exit / Errors	0 found, 1 not found, 2 error
Admin	No
Versions	Present since Windows Server 2003/Vista
Mistakes	`where` does not list built-ins (`where cd` fails); it shows files only
Security	Core tool for hijack and shadowing checks (Part XXI)
Lab / Cleanup	`where cmd`; then `where /q nosuchtool` and, as a separate line, `echo %ERRORLEVEL%`
Related	`path`, `assoc`, `ftype`
PowerShell	`Get-Command`, `(Get-Command x).Source`

Chapter 7. Variables, Flow and Process Commands

SET [Built-in, variables]

Field	Detail
Purpose / Why	Creates, shows, deletes and computes environment variables for the current CMD session
Syntax	`set [name[=value]]`, `set /a expression`, `set /p name=prompt`, `set "name=value"`
Key switches	`/a` arithmetic (32-bit signed integers), `/p` read a line of input
Beginner	`set GREETING=hello` then `echo %GREETING%`
Intermediate	`set /a TOTAL=5*4+2` then `echo %TOTAL%`
Advanced	`set "X=a b"` (quotes protect trailing spaces and special characters); substring `%X:~0,1%`; replace `%X:a=b%`
Real-world	Build file names with timestamps, parse options, hold counters in loops
Output	`set` alone lists all variables, alphabetical; `set P` lists those starting with P
Exit / Errors	1 when a named variable does not exist (`Environment variable X not defined`)
Admin	No. Changes only affect this session and its children. `setx` persists (Volume 2).
Versions	`/a` is 32-bit only; values beyond 2147483647 overflow silently
Mistakes	`set X = 5` creates a variable named `X ` (with space) holding `5`. `set /a` treats leading `0` as octal (`08` is an error).
Security	Variables are inherited by every child process; secrets placed in variables are readable by those children and in process environment blocks
Lab / Cleanup	`set A=1 & set A & set A= & set A` (last line reports not defined)
Related	`setlocal`, `setx`, `%VAR:~n,m%`
PowerShell	`$env:NAME = 'x'`, `Get-ChildItem Env:`

PATH [Built-in, command search]

Field	Detail
Purpose / Why	Shows or sets the list of folders CMD searches for external programs
Syntax	`path`, `path dir1;dir2;...`, `path %PATH%;C:\tools`
Beginner	`path`
Intermediate	`path %PATH%;C:\mytools` (this session only)
Advanced	`for %i in ("%PATH:;=" "%") do @echo %~i` (print one entry per line; delayed-expansion-free trick, test carefully)
Real-world	Temporarily exposing a toolkit folder to a script without editing system settings

Field	Detail
Output	`PATH=C:\Windows\system32;C:\Windows;...`
Exit / Errors	0
Admin	Session change no; permanent change (System Properties or `setx /m`) yes
Versions	Combined PATH length limits exist (2048 characters for `setx`); long paths are silently truncated by `setx`
Mistakes	Prepending an unsafe folder so it shadows system tools; trailing semicolons; unquoted entries containing spaces work but are fragile
Security	A writable folder early in PATH enables program shadowing (Part XXI)
Lab / Cleanup	Close the window to discard the change
Related	`where`, `setx`, `PATHEXT`
PowerShell	`$env:Path`

START [Built-in, process launch]

Field	Detail
Purpose / Why	Launches a program or document in a separate process or window, optionally without waiting
Syntax	`start ["title"] [/d dir] [/wait] [/min] [/b] [/low|/normal|/high] command [args]`
Key switches	`/wait` block until done, `/b` no new window, `/min`, `/d` working directory
Beginner	`start notepad`
Intermediate	`start "" "C:\Program Files\Some App\app.exe"`
Advanced	`start /wait "" cmd /c "script.cmd"` and then, on the next line, `echo %ERRORLEVEL%`
Real-world	Launching a GUI installer and waiting for it before the next script step
Output	New process/window; nothing printed
Exit / Errors	`The system cannot find the file ...`; with `/wait` the exit code is the child's
Admin	Does not elevate; the child inherits your token
Versions	Same
Mistakes	The **first quoted argument is treated as the window title**. `start "C:\path with spaces\a.exe"` does nothing useful. Always write `start "" "path"`.
Security	`start` on a document runs whatever the file association maps to; `start file.ext` resolves via ShellExecute rules, not just PATH
Lab / Cleanup	`start "" notepad` then close Notepad
Related	`call`, `cmd /c`, `ftype`
PowerShell	`Start-Process`

CALL [Built-in, script flow]

Field	Detail
Purpose / Why	Runs another batch file or a label in the same file and **returns** to the caller
Syntax	`call script.cmd args`, `call :label args`
Beginner	`call other.cmd`
Intermediate	`call :log "Starting"` (subroutine)
Advanced	`call echo %%VAR%%` forces a second round of percent expansion (Chapter 10)
Real-world	Modular batch: a library file of functions invoked with `call`
Output	Whatever the callee prints
Exit / Errors	`ERRORLEVEL` is the callee's last status; `The system cannot find the batch label specified`
Admin	No
Versions	Same
Mistakes	Running a second batch file **without** `call` transfers control permanently (it never returns). `call` doubles carets (^) and re-expands percents, which matters with untrusted data.
Security	The double-parse behavior of `call` is a classic injection amplifier (Part XX)
Lab / Cleanup	Create `sub.cmd` with `@echo in sub`, `.` then `sub` versus `call sub` in a two-line driver; delete both
Related	`goto`, `exit /b`, `setlocal`
PowerShell	`& .\script.ps1`

EXIT [Built-in, termination]

Field	Detail
Purpose / Why	Ends CMD, or with `/b` ends only the current batch file or subroutine
Syntax	`exit [/b] [code]`
Beginner	`exit`
Intermediate	`exit /b 3` (return 3 from the script)
Advanced	`exit /b %RC%` at the end of every function so callers can test `if errorlevel`
Real-world	Returning a meaningful status to a scheduler or CI job
Output	None
Exit / Errors	Sets the exit code
Admin	No
Versions	Same
Mistakes	Plain `exit` inside a script run from an open prompt **closes the window**. Use `exit /b`.
Security	Scripts that swallow failures (always exit 0) hide problems from monitoring

Field	Detail
Lab / Cleanup	`cmd /c exit 7` then `echo %ERRORLEVEL%` prints 7
Related	`call`, `goto :eof`
PowerShell	`exit`, `return`

CHOICE and TIMEOUT and PAUSE [External and Built-in, interaction and timing]

Field	Detail
Purpose / Why	Ask a key-press question, wait a number of seconds, or wait for any key
Syntax	`choice [/c keys] [/n] [/t sec /d key] [/m text]`; `timeout /t sec [/nobreak]`; `pause`
Beginner	`pause`
Intermediate	`choice /c YN /m "Continue"` then `if errorlevel 2 exit /b`
Advanced	`choice /c YN /t 10 /d N /m "Auto-decline in 10s"`
Real-world	Confirmation before a destructive step; polite retry delays
Output	`choice` prints `[Y,N]?` and returns the index of the key chosen (Y=1, N=2)
Exit / Errors	`choice` returns 1..n for the key position; 255 on error. `timeout` returns 0 or 1 if interrupted
Admin	No
Versions	`choice` and `timeout` ship with Windows Vista and later (they were add-ons on XP)
Mistakes	`if errorlevel 1` is true for **1 or higher**; test highest values first. `timeout` fails when stdin is redirected ("Input redirection is not supported"); use `ping -n 4 127.0.0.1 >nul` as a fallback.
Security	Unattended scripts with `pause` or `choice` hang forever and block schedulers
Lab / Cleanup	`timeout /t 5` then press a key to skip
Related	`set /p`, `if`
PowerShell	`Start-Sleep`, `Read-Host`, `$Host.UI.PromptForChoice`

Chapter 8. Text Filtering and Comparison Commands

FIND [External, text search]

Field	Detail
Purpose / Why	Simple fixed-string search in text
Syntax	`find [/v] [/c] [/n] [/i] "string" [file...]`
Key switches	`/v` lines not containing, `/c` count matching lines, `/n` line numbers, `/i` ignore case
Beginner	`find "error" log.txt`
Intermediate	`tasklist | find /i "chrome"`
Advanced	`type big.txt | find /c /v ""` (count lines)
Real-world	Count processes, filter command output inside scripts
Output	Matching lines; `---------- FILE` header when given file names
Exit / Errors	0 match, 1 no match, 2 error
Admin	No
Versions	Same; treats a file as ANSI (not UTF-16)
Mistakes	The search string **must** be in double quotes; `find` is not regex
Security	`find` is also a path-resolution target: the Unix-style `find` can shadow it on systems with Git tools installed; check `where find`
Lab / Cleanup	`echo abc> t.txt & find "b" t.txt & find /v "b" t.txt & del t.txt`
Related	`findstr`, `sort`
PowerShell	`Select-String`, `Where-Object`

FINDSTR [External, regex-like search]

Field	Detail
Purpose / Why	Searches with limited regular expressions, multiple patterns, recursive file search
Syntax	`findstr [/b] [/e] [/i] [/r] [/s] [/n] [/m] [/v] [/c:"str"] [/g:file] [/f:file] pattern files`
Key switches	`/i` case-insensitive, `/s` recurse, `/n` line numbers, `/m` file names only, `/r` regex (default for patterns), `/c:"a b"` literal phrase, `/v` invert
Beginner	`findstr /i "error" log.txt`
Intermediate	`findstr /s /i /m "password" *.config` (defensive audit for secrets in config files you own)
Advanced	`findstr /r /c:"^[0-9][0-9]*\.[0-9]" data.txt`

Field	Detail
Real-world	Searching a code base or logs for a pattern
Output	`file:line` when searching multiple files
Exit / Errors	0 found, 1 not found
Admin	No
Versions	Same. Regex is a small subset: `. * ^ $ [] [^] \< \> \x`. No `+`, no `?`, no alternation with `|`
Mistakes	Without `/c:` a space separates multiple patterns (OR). Backslash is an escape character. Fails on UTF-16 text.
Security	Quick scan of configuration folders for plaintext secrets is a defensive task; see Part XIX
Lab / Cleanup	`echo a b> t.txt & findstr "a c" t.txt & findstr /c:"a c" t.txt & del t.txt`
Related	`find`, `for /f`
PowerShell	`Select-String -Pattern -Path -Recurse`

SORT and CLIP [External, filtering]

Field	Detail
Purpose / Why	Sort lines of text; copy command output to the clipboard
Syntax	`sort [/r] [/+n] [/unique] [/m kb] [file]`; `clip < file`, `command | clip`
Beginner	`dir /b | sort`
Intermediate	`dir /b | sort /r`
Advanced	`sort /+20 list.txt` (sort by the text starting at column 20)
Real-world	Sorted reports; pasting diagnostics into a ticket (`ipconfig /all | clip`)
Output	Sorted lines; `clip` prints nothing
Exit / Errors	0
Admin	No
Versions	`clip` exists on Windows 7 and later; `sort /unique` needs Vista or later
Mistakes	`sort` sorts as **text**: `10` comes before `2`
Security	Clipboard contents can contain secrets and are visible to other programs in the session
Lab / Cleanup	`ipconfig /all | clip` then paste into Notepad
Related	`find`, `more`
PowerShell	`Sort-Object`, `Set-Clipboard`

FC and COMP [External, file comparison]

Field	Detail
Purpose / Why	Compare two files line by line (`fc`) or byte by byte (`comp`)
Syntax	`fc [/b] [/c] [/n] [/w] file1 file2`; `comp file1 file2`
Key switches	`fc /b` binary, `/c` ignore case, `/n` line numbers, `/w` compress whitespace
Beginner	`fc a.txt b.txt`
Intermediate	`fc /b before.bin after.bin`
Advanced	`fc /n baseline.txt current.txt > diff.txt` (configuration drift report)
Real-world	Verifying a copy or detecting configuration drift
Output	`FC: no differences encountered`, or differing blocks
Exit / Errors	0 identical, 1 differences, 2 error
Admin	No
Versions	Same
Mistakes	`fc` default mode depends on file extension (`.exe`, `.bin` compared in binary). For integrity proof use a hash (`certutil -hashfile`, Volume 3).
Security	`fc` equality is weaker evidence than a cryptographic hash
Lab / Cleanup	Make two similar files and diff them; delete them
Related	`certutil -hashfile`, `robocopy /l`
PowerShell	`Compare-Object`, `Get-FileHash`

Chapter 9. System Information Built-ins

VER, VOL, DATE, TIME, HOSTNAME [Built-in and External, basic identity]

Field	Detail
Purpose / Why	Report Windows version, volume label/serial, date, time, computer name
Syntax	`ver`; `vol [drive:]`; `date /t`; `time /t`; `hostname`
Beginner	`ver`, `hostname`, `date /t`, `time /t`
Intermediate	`vol C:`
Advanced	`echo %DATE% %TIME%` (dynamic variables, locale dependent)
Real-world	Stamping reports and logs
Output	`Microsoft Windows [Version 10.0.19045.xxxx]` on Windows 10; `10.0.22000+` is Windows 11
Exit / Errors	0
Admin	No. `date` **and** `time` **without** `/t` **prompt to change the clock** (requires rights); always add `/t`.
Versions	Windows 11 still reports major version 10.0; distinguish by build number (22000 or higher)
Mistakes	Parsing `%DATE%` in scripts on machines with different regional settings
Security	Accurate clocks matter for log correlation; tampering with time is an investigation signal
Lab / Cleanup	`ver & hostname & date /t & time /t`
Related	`systeminfo`, `wmic os` (deprecated), `wevtutil`
PowerShell	`Get-Date`, `[Environment]::OSVersion`, `Get-ComputerInfo`

ASSOC, FTYPE, CHCP, DOSKEY [Built-ins and External, configuration]

Field	Detail
Purpose / Why	Show file-type mappings; show or set the console code page; manage macros and history
Syntax	`assoc [.ext[=type]]`; `ftype [type[=command]]`; `chcp [codepage]`; `doskey [macro=text]`, `doskey /history`
Beginner	`assoc .txt` -> `.txt=txtfile`; `ftype txtfile`
Intermediate	`chcp` shows `Active code page: 437` (US OEM)
Advanced	`chcp 65001` switches the console to UTF-8 (output quality depends on the font and host)
Real-world	Diagnosing "wrong program opens this file type"; fixing garbled output in logs
Output	Mapping lines; `Active code page: nnn`
Exit / Errors	`File association not found for extension .xyz`
Admin	**Changing** `assoc`/`ftype` **requires elevation** and alters machine-wide behavior

Field	Detail
Versions	On modern Windows, per-user defaults (Open With, UserChoice) override `assoc`/`ftype`; the commands edit the machine-level keys only
Mistakes	Changing associations to "fix" something and breaking double-click behavior for all users
Security	File associations determine what runs when a document is opened; unexpected changes are an investigation signal
Lab / Cleanup	Read-only: `assoc .bat & ftype batfile`
Related	`reg query HKCR`, `start`, `mode`
PowerShell	`Get-ItemProperty HKCR:\.txt`, `[Console]::OutputEncoding`

Part III - CMD Parser Internals

Exactly how cmd.exe turns a line of text into running programs, phase by phase, with experiments you can verify yourself.

Chapter 10. The Phases of Command Interpretation

10.1 Why this chapter matters

Most "mysterious" batch behavior is not mysterious. CMD processes each line through a **fixed sequence of phases**, and each phase can only see what the previous phase produced. Once you know the sequence you can predict every result, and you can see exactly where untrusted input can change the meaning of a command (Part XX).

> **NOTE.** The phase model below is the widely used description derived from long-running community reverse-engineering of `cmd.exe` behavior (Microsoft does not publish the parser's internals). Every claim here is backed by an experiment you can run. When in doubt, run the experiment on your own build; that is the authoritative source.

10.2 The phases at a glance

```
Raw line (or a whole parenthesized block)
        |
Phase 1  PERCENT EXPANSION      %VAR%  %1  %*  %%  %~dp0
        |
Phase 2  SPECIAL CHARACTERS     quotes, ^ escapes, & && || | ( ) < >
        |                        tokenize, find redirection, build command
Phase 3  ECHO                   show the command if echo is on
        |
Phase 4  FOR VARIABLES          %%i / %i replaced inside a FOR body
        |
Phase 5  DELAYED EXPANSION      !VAR!   (only if enabled)
        |
Phase 6  REDIRECTION            open files for > >> < 2> ...
        |
Phase 7  CALL / restart         (only for CALL) carets doubled, phases repeat
        |
Phase 8  EXECUTE                built-in? or search for an external program
```

A **command block** (everything inside `( ... )`, or a full `if`/`for` body on one logical line) goes through Phases 1 and 2 **as a whole, once**, before any part of it runs. That single fact explains most of what confuses beginners.

10.3 Phase 1: percent expansion

CMD scans the line left to right for percent signs.

Text	Batch file result	Interactive prompt result
`%%`	one literal `%`	stays `%%`
`%NAME%` (defined)	its value	its value
`%NAME%` (undefined)	replaced by **nothing**	left as the literal text `%NAME%`
`%1` ... `%9`	that script argument	not special (literal)
`%*`	all script arguments	literal

Text	Batch file result	Interactive prompt result
`%~1, %~dp0`	argument with modifiers	literal
`%NAME:~2,3%`	substring	substring
`%NAME:a=b%`	replace text	replace text

Experiment 1 (interactive):

```
set TEST=hello
echo %TEST%
echo %NOSUCH%
echo 100%%
```

Expected output: `hello`, then `%NOSUCH%`, then `100%%`. In a batch file the last two lines print an empty line (with `ECHO is on.` for the bare `echo`) and `100%`.

Argument modifiers

Modifier	Meaning for `%~1` style
`%~1`	argument with surrounding quotes removed
`%~f1`	full path
`%~d1`	drive letter
`%~p1`	path without drive
`%~n1`	name only
`%~x1`	extension
`%~s1`	short (8.3) path
`%~a1`	file attributes
`%~t1`	last-modified time
`%~z1`	size in bytes
`%~dp0`	drive and path of the script itself (argument 0)

Modifiers combine: `%~nx1` is name plus extension.

10.4 Phase 2: special characters and tokenizing

This is the phase that decides what is **data** and what is **syntax**.

Special characters: `& && || | < > ( ) ^ " % !` (the last two are handled by other phases but matter here).

Rules:

1. A **double quote** toggles "quote mode." Inside quote mode, `& | < > ( )` lose their special meaning and are ordinary text. The quote characters themselves stay in the argument.
2. Outside quote mode, the **caret** `^` escapes the next character, so `^&` is a literal `&`. The caret itself is removed.
3. Inside quote mode a caret is an ordinary character and is **not** removed.
4. A caret as the last character of a line **joins the next line** (line continuation).

5. Unquoted `&`, `&&`, `||`, `|` separate commands; unquoted `>` `<` (with optional handle digit) are redirection; `(` and `)` open and close blocks.

Experiment 2:

```
echo a & echo b
echo a ^& echo b
echo "a & echo b"
echo ^^
echo "^^"
```

Expected: line 1 prints `a` then `b` (two commands). Line 2 prints `a & echo b` (one echo, caret escaped the ampersand). Line 3 prints `"a & echo b"` including the quotes. Line 4 prints `^`. Line 5 prints `"^^"` (carets untouched inside quotes).

10.5 The block-parse rule and the classic surprise

Experiment 3, save as `blk.cmd`:

```
@echo off
set X=1
if 1==1 (
    set X=2
    echo Percent sees: %X%
)
echo After the block: %X%
```

Output:

```
Percent sees: 1
After the block: 2
```

Explanation: the whole `if ... ( ... )` construct is one command. Phase 1 replaced `%X%` inside it **before** the block ran, when X was still 1. The `set X=2` did run, so after the block X really is 2.

The fix is **delayed expansion** (Phase 5), which expands `!X!` at the moment each inner command executes:

```
@echo off
setlocal EnableDelayedExpansion
set X=1
if 1==1 (
    set X=2
    echo Delayed sees: !X!
)
endlocal
```

Output: `Delayed sees: 2`.

DANGER / CAUTION. Never read `%ERRORLEVEL%` on the same line as the command that sets it (`cmd1 & echo %ERRORLEVEL%`). The whole line is expanded first, so you see the value from *before* `cmd1` ran. Put the `echo` on its own line, or use delayed expansion (`!ERRORLEVEL!`).

NOTE. The same effect happens with `for` loops. A counter incremented inside a `for` body and read with `%COUNT%` shows the value from before the loop. Use `!COUNT!` (with delayed expansion), or move the body into a subroutine invoked with `call`.

10.6 Phase 3: echo

If `echo` is on, CMD prints each command (after Phase 2) before running it. `@echo off` at the top of a script turns this off; the leading `@` hides the `echo off` line itself. Debugging a script often starts by removing `@echo off` and reading the echoed lines.

10.7 Phase 4: FOR variables

Inside a `for` body, `%i` (interactive) or `%%i` (batch) is replaced with the current item **after** Phase 3. That means a FOR variable can hold text that then goes through later phases (delayed expansion, CALL). Details of `for` itself are in Part V.

10.8 Phase 5: delayed expansion

Enabled with `setlocal EnableDelayedExpansion` inside a script, or `cmd /v:on` when starting a shell. When enabled, `!NAME!` is replaced when the command is about to execute.

Side effect: **an exclamation mark in ordinary text is now special**.

```
@echo off
echo Hello!
setlocal EnableDelayedExpansion
echo Hello!
echo Escaped: ^^!
```

The first `echo` prints `Hello!`; after delayed expansion is on, the second prints `Hello` (the lone `!` is consumed). To keep a literal `!` you must escape it, typically as `^^!` (the first caret is consumed in Phase 2, the second in Phase 5), so the third line prints `Escaped: !`.

Substring and replacement also work: `!NAME:~1,2!`, `!NAME:a=b!`.

10.9 Phase 6: redirection

Redirection is identified in Phase 2 and **executed** here. The tokens are removed from the command; the program never sees them. Details in Chapter 11.

10.10 Phase 7: CALL and the second pass

`call` restarts parsing so its arguments are expanded **twice**:

```
set NAME=Alice
set PTR=NAME
echo %%%PTR%%%
call echo %%%PTR%%%
```

Line 3 prints `%NAME%`. Line 4 prints `Alice`. Run this in a batch file. `call` also **doubles every caret** in its arguments, then restarts Phase 2. That doubling behavior is one reason `call` with untrusted data is dangerous (Part XX shows a safe pattern).

10.11 Phase 8: execution

CMD checks, in this order:

1. Is it a **doskey macro** (interactive only)?

2. Is it an **internal (built-in) command**?
3. Otherwise treat it as an **external program** and locate it (Chapter 12).

Once located, CMD calls the Windows API (`CreateProcess`) with the remaining text as the **command line**. The child program then parses that text **again, by its own rules** (Chapter 12). Two different parsers, two sets of quoting rules: another common source of bugs.

10.12 A worked trace

Consider this line in a script with `setlocal EnableDelayedExpansion`, where `N=world`:

```
echo Hello, %N% ^& !N!>out.txt
```

Phase	Result
1 percent	`echo Hello, world ^& !N!>out.txt`
2 special	Tokens: command `echo`; argument text `Hello, world & !N!`; redirect `> out.txt` (the caret escaped the ampersand)
3 echo	Shown if echo is on
5 delayed	`!N!` becomes `world`
6 redirect	Open `out.txt` for writing
8 execute	Built-in `echo` writes `Hello, world & world` (note: no space before > so no trailing space)

Lab 41. Watching the phases with echo

Objective. Observe percent-expansion timing inside a block. **Difficulty.** [B]. **Prerequisites.** Chapters 1-3 and Section 10.5. **Environment.** Any Windows 10/11 prompt, scratch folder.

Commands. Type these one at a time. Interactively, an undefined `%X%` is left as literal text, so it lands in the file unchanged (run `set X=` first to be sure X is undefined).

```
mkdir "%TEMP%\cmdlab" & cd /d "%TEMP%\cmdlab"
set X=
> l41.cmd echo @echo on
>> l41.cmd echo set X=1
>> l41.cmd echo if 1==1 (
>> l41.cmd echo    set X=2
>> l41.cmd echo    echo X is %X%
>> l41.cmd echo )
type l41.cmd
l41.cmd
```

Expected result. `type` shows the six generated lines. Running the script echoes each command and prints `X is 1`, even though `set X=2` ran just before it. **Explanation.** Phase 1 expanded `%X%` for the whole `if` block before any of it executed, when X was 1. **Verification.** Run `echo %X%` afterwards; it prints `2`, proving the `set` did run. Now insert `setlocal EnableDelayedExpansion` as the first line and change `%X%` to `!X!` (open the file in Notepad); the output becomes `X is 2`. **Troubleshooting.** If the file lacks the `echo` lines, check that you did not put a space before > in a way that added trailing spaces; view with `type`. **Cleanup.** `del l41.cmd`. **Challenge.** Predict the output of a script that sets `N=0` and runs `for /l %%i in (1,1,3) do (set /a N+=1 >nul & echo N is %N%)`. **Solution.** It prints `N is 0` three times because `%N%` was expanded once before the loop; with

delayed expansion and `!N!` it prints 1, 2, 3.

Lab 42. Caret and quote experiments

Objective. Learn when carets and quotes are consumed. **Difficulty.** [B]. **Prerequisites.** Lab 41. **Environment.** Interactive prompt.

Commands.

```
echo a ^& b
echo "a & b"
echo ^^
echo "^^"
echo a^
b
```

Expected result. `a & b`; `"a & b"`; `^`; `"^^"`; then a two-line continuation prints `ab`. **Explanation.** Unquoted caret escapes the next character; in quotes it is literal; a caret at end of line joins lines. **Verification.** Compare with the predictions in Section 10.4. **Troubleshooting.** If the last example asks `More?`, that is CMD waiting for the continued line (type `b` and press Enter). **Cleanup.** None. **Challenge.** Predict `echo ^"a & b^"`. **Solution.** The escaped quotes do not start quote mode, so `&` is an operator: prints `"a` then tries to run `b"` and fails with `'b"' is not recognized`.

Chapter 11. Operators, Redirection, Pipes and Nested Commands

11.1 Command separators

Operator	Meaning	Example
`a & b`	Run a, then b, always	`echo one & echo two`
`a && b`	Run b **only if a succeeded** (exit 0)	`mkdir x && cd x`
`a || b`	Run b **only if a failed** (nonzero)	`cd x || echo missing`
`a | b`	Send a's stdout into b's stdin	`dir /b | sort`
`( ... )`	Group commands as one unit	`(echo a & echo b) > out.txt`

Combined: `cmd1 && cmd2 || cmd3` is **not** if-then-else. If `cmd1` succeeds but `cmd2` fails, `cmd3` also runs. Use a real `if ... else` when that matters.

Experiment:

```
cd nosuchdir && echo yes || echo no
echo start & (echo A & echo B) & echo end
```

Expected: `no` (after the cd error message), then `start`, `A`, `B`, `end`.

11.2 The three standard streams

Stream	Handle number	Default target
stdin	0	keyboard
stdout	1	screen
stderr	2	screen

11.3 Redirection operators

Operator	Effect
`> file`	stdout to file, **overwrite**
`>> file`	stdout to file, **append**
`< file`	file becomes stdin
`2> file`	stderr to file
`2>> file`	stderr appended
`2>&1`	send stderr to wherever stdout currently points
`1>&2`	send stdout to where stderr currently points
`> nul`	discard output (`nul` is the null device)

Order matters

Redirections are applied **left to right**.

```
dir nosuch > all.txt 2>&1
dir nosuch 2>&1 > all2.txt
```

Line 1 sends stdout to the file first, then points stderr at the same place: **both** end up in `all.txt`. Line 2 first points stderr at the *current* stdout (the screen), then moves stdout to the file: the error message appears on screen and `all2.txt` gets only the standard output (empty here).

The digit trap

If a **single digit stands alone as a token immediately before** `>`, CMD treats it as a handle number rather than text. This bites most often when a variable happens to hold a digit:

```
set N=2
echo %N%>t.txt
```

After percent expansion the line is `echo 2>t.txt`, which redirects **stderr** (handle 2) to the file. The echo then has no argument, so it prints `ECHO is on.` on screen and `t.txt` stays empty. Safe patterns: put the redirection first (`>t.txt echo %N%`), or add a space and accept the trailing space in the file (`echo %N% >t.txt`). A digit that is glued to a longer word is reported in published analyses as ordinary text; Lab 52 has you verify that on your own build.

Redirect placement

Redirection can appear at the start of a command: `>log.txt echo hello`. That style avoids the digit trap and trailing-space problem entirely.

11.4 Pipes

`a | b` starts **two new** `cmd.exe` **processes**, one for each side, connected by an anonymous pipe. Consequences:

1. Environment changes on either side are **lost** when they finish.
2. Both sides run at once.
3. Each side goes through the parser again in the child.

Experiment:

```
set V=orig
set V=changed | more
echo %V%
```

Expected output: `orig`. The `set` ran in a child process.

> **NOTE.** The same reason applies to `for /f ... in ('command')`: the command runs in a child `cmd /c`, so its environment changes do not persist.

11.5 cmd /c and cmd /k

Form	Behavior
`cmd /c command`	Run command, then **terminate** the new CMD

Form	Behavior
`cmd /k command`	Run command, then **stay** open at a prompt
`cmd /s /c "..."`	Strip the outer quote pair exactly, then run
`cmd /v:on /c ...`	Enable delayed expansion for that run
`cmd /q`	Echo off
`cmd /d`	Skip AutoRun registry commands

Quote rules for `cmd /c`: if the first character after `/c` is a quote and several other conditions hold (exactly two quotes, no special characters between them, the string names an executable file), CMD keeps them. Otherwise it strips the **first and last** quote of the line. That is why `cmd /c "a" "b"` and `cmd /c ""a" "b""` behave differently. To avoid guessing, use `cmd /s /c "..."` and wrap the whole command in one outer pair.

Exit code: `cmd /c` returns the exit code of the command it ran.

```
cmd /c exit 5
echo %ERRORLEVEL%
```

Expected: `5`.

11.6 Nesting and precedence

Rough precedence, tightest first: redirections, then `|`, then `&&` and `||` (equal precedence, left to right), then `&`. Parentheses override.

```
echo a & echo b | findstr b
```

The pipe binds first: it pipes only `echo b` to findstr, so output is `a` then `b`.

11.7 Interactive versus batch differences

Behavior	Interactive	Batch file
Undefined %VAR%	left unchanged	removed
%%	stays %%	becomes %
FOR variable	`%i`	`%%i`
Echo	on by default	on unless `@echo off`
%1..%9, %*	literal	script arguments
Ctrl+C	stops the command	asks `Terminate batch job (Y/N)?`
`exit`	closes the window	closes the window unless `/b`
`pause`	waits at the prompt	same
`setlocal`/`endlocal`	no effect at prompt	scoping
`:label` and `goto`	not available at the prompt	supported

Lab 43. Redirection order

Objective. Prove that redirection order changes where output goes. **Difficulty.** [I]. **Prerequisites.** Section 11.3. **Environment.** Scratch folder.

Commands.

```
dir nosuch > a.txt 2>&1
dir nosuch 2>&1 > b.txt
type a.txt
type b.txt
```

Expected result. After line 1, nothing prints and `a.txt` holds the error text. Line 2 prints the error on screen; `b.txt` is empty. **Explanation.** `2>&1` copies the *current* target of stdout. **Verification.** `dir a.txt b.txt` shows a non-zero size for `a.txt` and 0 bytes for `b.txt`. **Troubleshooting.** On some builds the `File Not Found` text is `The system cannot find the path specified.`; either proves the point. **Cleanup.** `del a.txt b.txt`. **Challenge.** Send stdout to `out.txt` and stderr to `err.txt` in one line. **Solution.** `dir nosuch > out.txt 2> err.txt`.

Lab 44. Pipes create child processes

Objective. Show that piped commands run in separate CMD instances. **Difficulty.** [I]. **Prerequisites.** Section 11.4. **Environment.** Interactive prompt.

Commands.

```
set V=orig
set V=changed | more
echo %V%
set V=changed
echo %V%
```

Expected result. `orig` then `changed`. **Explanation.** The piped `set` ran in a child process whose environment vanished. **Verification.** Run `set V=x | rem` and check `set V`. **Troubleshooting.** If you see `changed` first, you typed the wrong line; re-run from the top. **Cleanup.** `set V=`. **Challenge.** Explain why `for /f %i in ('set V=z') do @echo %i` also does not change V. **Solution.** The command inside quotes runs in a child `cmd /c`.

Lab 45. Conditional chaining is not if-else

Objective. Demonstrate the `&&`/`||` pitfall. **Difficulty.** [I]. **Prerequisites.** Section 11.1. **Environment.** Interactive prompt.

Commands.

```
echo ok && cmd /c exit 1 || echo ELSE-branch
echo ok && echo also-ok || echo ELSE-branch
cmd /c exit 1 && echo A || echo B
```

Expected result. Line 1 prints `ok` then `ELSE-branch`; line 2 prints `ok`, `also-ok` only; line 3 prints `B`. **Explanation.** `||` tests the exit status of the last command in the chain, not the first. **Verification.** Re-order and predict. **Troubleshooting.** `echo` always succeeds; use `cmd /c exit N` to fake failures. **Cleanup.** None. **Challenge.** Rewrite line 1 as a proper `if errorlevel` structure. **Solution.** Store the status: `echo ok & cmd /c exit 1 & if errorlevel 1 (echo failed) else (echo passed)`.

Chapter 12. Command Lookup, PATH, PATHEXT and Argument Parsing

12.1 What happens when you type a name

```
you type:  ping
   |
doskey macro named ping?      (interactive only)  -> expand it
   |
internal command named ping?  (cd, dir, set, ...) -> run it inside cmd.exe
   |
contains a path or drive?     (C:\x\ping.exe, .\ping) -> try exactly that
   |
otherwise search:  current directory, then each PATH folder, in order
      in each folder, try the name + each extension in PATHEXT, in order
   |
found -> CreateProcess;  not found -> "'ping' is not recognized ..."
```

PATHEXT

`PATHEXT` lists the extensions CMD will try when you omit one:

```
echo %PATHEXT%
```

Typical value: `.COM;.EXE;.BAT;.CMD;.VBS;.VBE;.JS;.JSE;.WSF;.WSH;.MSC`. The order matters: if `tool.com` and `tool.exe` and `tool.bat` all exist in the same folder, `tool` runs `tool.com`.

Built-ins beat files

Because internal commands are checked first, dropping `dir.exe` on the PATH does **not** replace `dir`. But typing `dir.exe` explicitly (with an extension) skips the internal command check and would run that file. External tools such as `find`, `sort`, `ping`, `whoami` have no such protection: whichever matching file CMD finds first wins.

The current directory is searched first

When CMD runs an external command without a path, it looks in the **current directory before PATH**. A hostile or accidental file named like a system tool, sitting in the folder you happen to be in, can be executed instead. Setting the environment variable `NoDefaultCurrentDirectoryInExePath` (to any value) tells CMD and the Windows path-search API to stop searching the current directory first; test the effect on your own build in the lab below. PowerShell, by contrast, does not run a command from the current directory without `.\`.

What `where` shows

```
where ping
where find
where python
```

The **first** line of each is what would run, subject to the current-directory rule (which `where` also follows). If `where find` lists a path outside `C:\Windows\System32`, a different `find` is shadowing the Windows one.

Launching by file association

If you type a document name (`report.docx`) CMD hands it to the shell, which looks up the file association and runs the associated program. `start` does the same explicitly. That resolution comes from the registry, not from

PATH.

12.2 Two parsers, two sets of rules

After CMD finishes (Chapter 10) it builds a **single command-line string** and gives it to Windows. The launched program parses that string itself. Most C/C++ programs use the Microsoft C runtime's rules (equivalent to `CommandLineToArgvW`):

In the command line	Program sees
`a b c`	three arguments: `a`, `b`, `c`
`"a b" c`	two arguments: `a b`, `c`
`a\b`	one argument `a\b` (backslashes literal unless before a quote)
`a\"b`	one argument `a"b` (backslash escapes the quote)
`"a\\" b`	`a\` then `b` (two backslashes before a quote collapse to one, and the quote closes)

Not every program follows those rules. Installers, `cmd.exe` itself, `msiexec`, `findstr` and many legacy tools do their own parsing. When a tool "eats" your quotes, this is usually why. Two practical rules:

1. Use **double quotes** around any path or value containing spaces.
2. Do not put a **trailing backslash** before the closing quote: `"C:\dir\"` can be read as an escaped quote. Write `"C:\dir\\"` or `"C:\dir."` or use a path without the final backslash.

12.3 Environment inheritance

Every process starts with a **copy** of its parent's environment block and current directory. Changes a child makes never propagate back. That is why `setlocal` exists in scripts, why a batch file cannot change your shell's variables unless it is run inside the same CMD (not via `cmd /c`), and why `setx` (which writes to the registry for future processes) does not change the current window.

Lab 46. Who answers to a name?

Objective. Resolve names with `where` and check built-ins. **Difficulty.** [B]. **Prerequisites.** Chapter 12. **Environment.** Any prompt.

Commands.

```
where ping
where find
where cd
echo %PATHEXT%
dir /b %WINDIR%\System32\ping.*
```

Expected result. `where ping` prints `C:\Windows\System32\ping.exe`; `where cd` reports `INFO: Could not find files for the given pattern(s).` because `cd` is internal; PATHEXT prints the extension list; the last command lists `PING.EXE`. **Explanation.** Only files appear in `where`; built-ins live inside `cmd.exe`. **Verification.** `where /q cd` followed, on its own line, by `echo %ERRORLEVEL%` prints `1`. **Troubleshooting.** If `where` lists a second `find` (for example from Git), note that the first listed is the one that runs. **Cleanup.** None. **Challenge.** List every PATH folder on its own line. **Solution.** `for %i in ("%PATH:;=" "%") do @echo %~i`

at the prompt (double the percent signs in a script).

Lab 47. Current-directory shadowing (safe demonstration)

Objective. See that a harmless look-alike in the current folder is found before the real tool. **Difficulty.** [A]. **Prerequisites.** Lab 46. **Environment. VM only.** Scratch folder.

Commands.

```
mkdir "%TEMP%\cmdlab\shadow" & cd /d "%TEMP%\cmdlab\shadow"
> hostname.cmd echo @echo I am a harmless script named hostname
where hostname
hostname
hostname.exe
set NoDefaultCurrentDirectoryInExePath=1
hostname
where hostname
set NoDefaultCurrentDirectoryInExePath=
```

Expected result (verify on your build). `where hostname` lists the local `hostname.cmd` first and `C:\Windows\System32\hostname.exe` second. The plain `hostname` prints the impostor message, because the current directory is searched first. `hostname.exe` prints the real computer name (no local `.exe` exists). With the environment variable set, plain `hostname` should print the real name. **Explanation.** The search visits the current directory first, trying each PATHEXT extension there before moving to the next folder. **Verification.** Compare `where` output with and without the variable. **Troubleshooting.** If your result differs from this prediction, record your Windows build (`ver`) and what you saw; documenting differences is the point of the lab. **Cleanup.** `cd /d %TEMP% & rmdir /s /q cmdlab\shadow`. **Challenge.** Name two ways to be immune. **Solution.** Use an absolute path such as `%SystemRoot%\System32\hostname.exe`, or call a built-in (built-ins are matched before any file search).

Part IV - CMD Edge Cases

The places where CMD behaves in ways nobody would guess: spaces, quotes, special characters, encodings, line endings, and the timing of expansion. Every item has an experiment.

Chapter 13. Expert Edge-Case Catalog

13.1 Spaces in paths

Unquoted spaces split arguments.

```
copy C:\Program Files\app.ini D:\backup
```

CMD sees three arguments. Always write:

```
copy "C:\Program Files\app.ini" "D:\backup\"
```

For `if exist` and `set`, quote the whole thing: `if exist "%ProgramFiles%\App"`, `set "DIR=C:\Program Files"`. With FOR and arguments, `%~1` removes the quotes so you can put them back deliberately: `"%~1"`.

13.2 Quotes inside quotes

CMD has **no escape for a double quote** inside quote mode. Each `"` toggles the mode. The quote characters are passed on to the program, and the **program's** rules decide what they mean. Consequences:

```
echo "He said "hi" now"
```

prints the text unchanged, but which `&`, `|`, `<`, `>` are active depends on whether an even or odd number of quotes precede them. A quote count that goes odd in the middle of a line (a quote in a variable value, for instance) silently flips what is "inside quotes" for the rest of the line. That mismatch is the root of many injection bugs (Part XX).

13.3 Trailing spaces

Everything after `=` in `set` is the value, **including trailing spaces**.

```
set NAME=Alice
echo [%NAME%]
set "NAME=Alice"
echo [%NAME%]
```

The first prints `[Alice ]`, the second `[Alice]`. The quoted form `set "NAME=value"` is the recommended style, and it also ignores anything typed after the closing quote. The same applies to `echo text > file`: the space before `>` becomes part of the file's first line.

13.4 Special characters as data

`& | < > ^ ( ) % ! "` are syntax. To hold them as data:

Situation	How
Plain command, one character	caret: `echo a ^& b`
Assigning a value	`set "S=a&b"` (quotes stop `&` being an operator during assignment)
Using the value	use **delayed expansion**: `echo !S!`
Literal `%` in batch	`%%`

Situation	How
Literal `!` with delayed expansion	`^^!`

Experiment (interactive):

```
set "S=a&b"
echo %S%
```

Line 2 prints `a` and then complains that `b` is not recognized, because Phase 1 pasted the value into the line **before** Phase 2 parsed it, turning `&` into an operator. With delayed expansion (`echo !S!` inside a script with `setlocal EnableDelayedExpansion`, see Lab 50) the value is inserted **after** parsing, so `&` stays data and the output is `a&b`. This asymmetry (expansion before versus after parsing) is the key to safe handling of untrusted strings.

13.5 Parentheses

A `)` ends the current block. Inside a block, a literal `)` must be escaped or quoted.

```
if 1==1 (echo Program Files (x86))
```

The first `)` closes the block early, producing truncated output or a syntax error. Correct forms:

```
if 1==1 (echo Program Files ^(x86^))
if 1==1 (echo "Program Files (x86)")
```

Paths held in variables such as `%ProgramFiles(x86)%` are a notorious trigger: expanding them inside a block injects `)` characters. Use delayed expansion or assign to a variable outside the block.

13.6 Redirection and pipe quirks

- A digit standing alone as a token immediately before `>` is a handle number (`echo %N%>f` when N is 2).
- `|` runs both sides in child CMD instances, so `|` with a built-in like `set` does not keep the change.
- `type file | command` is an unnecessary extra process; `command < file` is lighter.
- `>nul` discards stdout; `2>nul` discards errors; `>nul 2>&1` discards both.

13.7 FOR variable details

- FOR variables are **single letters**, and `%%i` and `%%I` are **different** variables.
- In batch use `%%i`; at the prompt use `%i`.
- `%%~nxi` applies the same modifiers as arguments (`~n`, `~x`, `~f`, `~d`, `~p`, `~t`, `~z`).
- An empty FOR set (for example `*.xyz` with no matches) runs the body **zero** times.

13.8 CALL expansion

`call` re-runs percent expansion and doubles carets (Chapter 10). Practical effects:

You write	What happens
`call echo %X%`	%X% expanded at Phase 1 of the first pass; any % in X's value is expanded again
`call echo a^&b`	caret consumed on the first pass; & becomes an operator on the second pass

You write	What happens
`call :sub "%~1"`	quoting protects most specials, but a % or ^ inside can still misbehave

13.9 Code pages and encodings

Windows consoles use an **OEM code page** (437 in the US) and GUI programs typically use an **ANSI code page** (1252 in Western Europe/US). Text produced by CMD built-ins and redirected to a file uses the console's output code page; Notepad may interpret it differently, so accented characters can appear wrong.

```
chcp
chcp 65001
chcp 437
```

`chcp 65001` switches to UTF-8 for console I/O. Quality depends on the host and font, and some legacy tools still misbehave under 65001. Settings > Time & language > Language & region > Administrative language settings > **Beta: Use Unicode UTF-8 for worldwide language support** changes the system-wide ANSI code page to UTF-8 (Windows 10 1903+); it can break older applications, so test first.

Batch files should be saved as ANSI (or UTF-8 **without** a byte-order mark). A UTF-8 **BOM** at the start of a script can make the first line fail with a strange `'...@echo' is not recognized` message on some builds.

`cmd /u` makes CMD built-ins write **UTF-16LE** to pipes and files, while `cmd /a` uses ANSI. FOR /F and `find`/`findstr` do not read UTF-16 text files correctly; convert first (`type utf16.txt > ansi.txt` often works because `type` converts in console mode, but verify).

13.10 Unicode limitations

CMD built-ins are not fully Unicode-aware. Filenames with characters outside the active code page may show as `?`. For Unicode-heavy work use PowerShell or a real language; use CMD only to call it.

13.11 Line endings (CRLF)

Batch files should use **CRLF** line endings. A file with bare LF endings can make `goto` and `call :label` fail to find labels or execute lines oddly (the classic failure involves labels that straddle internal read boundaries). If a script that looks fine behaves erratically, check line endings.

```
find /c /v "" script.cmd
```

counts lines; compare to what your editor shows, and use an editor that can display line endings.

13.12 Standard input behavior

- `set /p X=prompt` reads **one line** from stdin; with `< file` it reads the first line.
- `more`, `sort`, `find` read stdin until end-of-file.
- Pressing **Ctrl+Z, Enter** at the keyboard sends end-of-file.
- `pause < nul` does not wait (input is already at EOF), a useful way to disable pauses in automated runs.
- `echo.| command` feeds a single Enter key to something that asks "press Enter."

13.13 Interactive versus batch: expansion-timing summary

Question	Answer
When is %VAR% replaced?	Before the command (or entire block) is parsed
When is !VAR! replaced?	Just before that individual command runs
When is %%i replaced?	When the FOR body runs, per iteration
When is %1 replaced?	With %VAR%, at Phase 1
When are redirections opened?	Just before execution, after delayed expansion

Lab 48. Spaces in paths

Objective. Reproduce and fix the unquoted-space failure. **Difficulty.** [B]. **Prerequisites.** Section 13.1. **Environment.** Scratch folder.

Commands.

```
mkdir "my folder"
> "my folder\a b.txt" echo data
copy my folder\a b.txt backup.txt
copy "my folder\a b.txt" backup.txt
type backup.txt
```

Expected result. Line 3 fails (`The system cannot find the file specified.` or a syntax error, depending on how the pieces are interpreted); line 4 prints `1 file(s) copied.`; `type` prints `data`. **Explanation.** Unquoted spaces split the path into several arguments. **Verification.** `dir /b backup.txt`. **Troubleshooting.** If the copy succeeded on line 3, a file named `my` already existed. **Cleanup.** `del backup.txt & rmdir /s /q "my folder"`. **Challenge.** List the file correctly with `dir`. **Solution.** `dir "my folder\a b*"`.

Lab 49. Trailing space trap

Objective. Prove that `set` keeps trailing spaces. **Difficulty.** [B]. **Prerequisites.** Section 13.3. **Environment.** Interactive prompt.

Commands.

```
set NAME=Alice
echo [%NAME%]
set "NAME=Alice"
echo [%NAME%]
if "%NAME%"=="Alice" (echo match) else (echo no match)
```

Expected result. `[Alice ]`, `[Alice]`, `match`. **Explanation.** The unquoted form includes the space typed after `Alice`; the quoted form does not. **Verification.** Use `set NAME=Alice ` then test the `if`: it prints `no match`. **Troubleshooting.** Some editors strip trailing spaces before you can see them; type the line yourself. **Cleanup.** `set NAME=`. **Challenge.** Make a script safe against trailing spaces in a user-supplied argument. **Solution.** `set "ARG=%~1"` and trim with a loop, or require quoted assignment everywhere.

Lab 50. Data versus syntax: & in a value

Objective. Show why delayed expansion is safer for untrusted text. **Difficulty.** [A]. **Prerequisites.** Sections 10.5, 13.4. **Environment.** Scratch folder.

Commands. Save as `l50.cmd`:

```
@echo off
setlocal
set "S=a&echo INJECTED"
echo Percent: %S%
setlocal EnableDelayedExpansion
echo Delayed: !S!
endlocal
```

Run `l50.cmd`.

Expected result. `Percent: a` followed by `INJECTED`, then `Delayed: a&echo INJECTED`. **Explanation.** The value was pasted into the line before parsing, so `&` became an operator and a second `echo` ran. Delayed expansion inserts the text after parsing, so it stays inert. **Verification.** Replace `a&echo INJECTED` with `a&echo HI` and repeat. **Troubleshooting.** If both lines look identical, check that the `setlocal EnableDelayedExpansion` line is before the second echo. **Cleanup.** `del l50.cmd`. **Challenge.** Explain why `echo !S!` can still be dangerous when passed onward to `call` or `cmd /c`. **Solution.** The second parser re-interprets the text; you must quote or validate before handing it on (Part XX).

Lab 51. Parentheses in blocks

Objective. Trigger and fix the `)` problem. **Difficulty.** [I]. **Prerequisites.** Section 13.5. **Environment.** Scratch folder.

Commands. Save as `l51.cmd`:

```
@echo off
set "P=C:\Program Files (x86)"
if 1==1 (
    echo Unsafe: %P%
)
setlocal EnableDelayedExpansion
if 1==1 (
    echo Safe: !P!
)
endlocal
```

Expected result. The first block misbehaves: truncated text and/or a syntax error such as `) was unexpected at this time.` (the exact symptom varies by build, and the error stops the script). Delete the first block and run again: the second prints `Safe: C:\Program Files (x86)`. **Explanation.** The value's `)` closes the block when pasted in Phase 1. **Verification.** Quote the first echo as `echo Unsafe: "%P%"` and compare. **Troubleshooting.** The exact error text varies; any syntax error confirms the effect. **Cleanup.** `del l51.cmd`. **Challenge.** Do it without delayed expansion. **Solution.** `call echo Safe: %%P%%` or move the echo into a subroutine that uses `%~1`.

Lab 52. Redirect digit trap

Objective. Observe how a lone digit before > becomes a handle number. **Difficulty.** [I]. **Prerequisites.** Section 11.3. **Environment.** Scratch folder.

Commands.

```
set N=2
echo %N%>t52.txt
type t52.txt
echo %N% >t52.txt
dir /b t52.txt & type t52.txt
> t52.txt echo %N%
type t52.txt
echo value%N%>t52.txt
type t52.txt
```

Expected result. Line 2 prints `ECHO is on.` on screen and creates an **empty** file (stderr was redirected, echo had no argument), so the first `type` prints nothing. Line 4 writes `2 ` (with a trailing space) to the file. Line 6 writes `2`. The last pair tests the glued-digit case: published analysis says the file should contain `value2`; record what your build does. **Explanation.** A digit that is a separate token right before > is a handle. **Verification.** `dir t52.txt` shows size 0 after line 2. **Troubleshooting.** If line 2 wrote `2` to the file, re-check that there was no space before >. **Cleanup.** `del t52.txt & set N=`. **Challenge.** Write the literal text `1>2` to a file. **Solution.** `> t52.txt echo 1^>2`.

Lab 53. Code pages and accented text

Objective. See how the console code page affects output files. **Difficulty.** [I]. **Prerequisites.** Section 13.9. **Environment.** Scratch folder, **VM**.

Commands. Type an accented letter (for example e with an acute accent, pasted in or typed with Alt+0233) where shown:

```
chcp
> t53a.txt echo cafe-with-accent
chcp 65001
> t53b.txt echo cafe-with-accent
chcp 437
certutil -dump t53a.txt
certutil -dump t53b.txt
```

Replace the word `cafe-with-accent` with the accented spelling each time, open both files in Notepad, and compare.

Expected result. The two files contain different bytes for the accented letter (a single byte under an OEM page such as 437, two bytes under UTF-8), and Notepad may display one of them incorrectly. **Explanation.** Bytes are the same; interpretation differs. **Verification.** The two `certutil -dump` outputs show the actual bytes. **Troubleshooting.** If the font shows boxes, choose a font such as Consolas that contains the glyph. **Cleanup.** `chcp` back to your original value and `del t53a.txt t53b.txt`. **Challenge.** Write a UTF-8 file from CMD without a BOM. **Solution.** Under `chcp 65001`, > from `echo` writes UTF-8 bytes without a BOM on current builds; verify with the hex dump.

Lab 54. Line-ending inspection

Objective. Detect CRLF versus LF. **Difficulty.** [I]. **Prerequisites.** Section 13.11. **Environment.** Scratch folder.

Commands.

```
> crlf.txt echo line1
>> crlf.txt echo line2
certutil -dump crlf.txt
```

Expected result. The dump shows `0d 0a` after each line (bytes `0d 0a`). **Explanation.** `echo` writes CRLF. **Verification.** Open the file in an editor that can convert to LF, save as `lf.txt`, then `certutil -dump lf.txt` shows only `0a`. **Troubleshooting.** `certutil -dump` output is long for big files; use small test files. **Cleanup.** `del crlf.txt lf.txt`. **Challenge.** Write a script that warns when a batch file has no `0d`. **Solution.** `findstr /r /c:"\r" file` is not reliable in CMD; use `certutil -dump` output and `find "0d 0a"` as a first check, or use PowerShell for robust scanning.

Lab 55. Expansion timing table

Objective. Predict-then-verify exercise covering Phases 1, 4 and 5 together. **Difficulty.** [A]. **Prerequisites.** Chapter 10. **Environment.** Scratch folder.

Commands. Save as `l55.cmd`; **predict every output line before running it**.

```
@echo off
setlocal EnableDelayedExpansion
set N=0
for %%i in (a b c) do (
    set /a N+=1 >nul
    echo pct=%N%  delayed=!N!  item=%%i
)
echo after: %N%
set "LIST="
for %%i in (x y z) do set "LIST=!LIST!%%i;"
echo !LIST!
endlocal
```

Expected result. Three lines `pct=0 delayed=1 item=a`, `pct=0 delayed=2 item=b`, `pct=0 delayed=3 item=c`, then `after: 3`, then `x;y;z;`. **Explanation.** `%N%` inside the body was expanded once in Phase 1 (value 0); `!N!` is live. After the loop Phase 1 sees the final value 3. The second loop builds a string with a delayed-expansion accumulator. **Verification.** Remove `EnableDelayedExpansion` and see `!N!` printed literally. **Troubleshooting.** If `pct` shows 1, 2, 3 you have a stale `N` from earlier; use `set N=0` (already present) and run in a fresh prompt. **Cleanup.** `del l55.cmd`. **Challenge.** Rewrite so `pct` counts correctly without delayed expansion. **Solution.** Move the loop body into a subroutine: `for %%i in (a b c) do call :body %%i` and in `:body` use `set /a N+=1` then `call echo pct=%%N%%`.

Part V - Batch Scripting

Batch files as a real (if quirky) programming language: variables, arguments, decisions, loops, FOR /F parsing, subroutines, errors, logging and testing.

Chapter 14. Your First Scripts: Variables and Arguments

14.1 What a batch file is

A **batch file** is a plain text file with the extension `.bat` or `.cmd` whose lines CMD runs in order. The two extensions are run by the same interpreter. The one documented difference is small: after certain built-in commands (such as `set`, `path`, `prompt`, `assoc`, `ftype`) a `.cmd` file resets `ERRORLEVEL` on success, while a `.bat` file leaves it unchanged. New work should use `.cmd`.

Create your first script with Notepad (Save as type: **All Files**, encoding **ANSI** or UTF-8 without BOM), name it `hello.cmd`:

```
@echo off
echo Hello World
pause
```

Line by line: `@echo off` stops CMD from printing each command (the `@` hides that line itself). `echo Hello World` prints text. `pause` prints `Press any key to continue . . .` and waits.

Run it from a prompt with `hello` or `hello.cmd`, or double-click it (the window closes immediately unless something waits, which is why beginners add `pause`).

> **NOTE.** Always use a **standard-user** prompt to run scripts you just wrote; escalate only when a step truly needs it.

14.2 A professional script header

```
@echo off
rem ============================================================
rem  name   : backup_docs.cmd
rem  purpose: Copy Documents to a dated folder (example)
rem  usage  : backup_docs.cmd [destination]
rem ============================================================
setlocal EnableExtensions
cd /d "%~dp0"
```

- `setlocal` makes variable changes **local** to the script (they vanish when it ends). `EnableExtensions` guarantees the extended command syntax used in this book.
- `cd /d "%~dp0"` moves to the script's own folder so relative paths behave regardless of where it was launched.
- `rem` starts a comment. `::` also works as a comment but breaks inside parenthesized blocks; prefer `rem`.

14.3 Variables

```
set "NAME=Alice"
set /a COUNT=3*4
echo Hello %NAME%, count=%COUNT%
set /p ANSWER=Type something:
echo You typed: %ANSWER%
```

- Always write `set "NAME=value"` (quotes protect special characters and trailing spaces).
- Variable names are **case-insensitive** (`name` and `NAME` are the same); values are text.
- `set /a` does integer math: `+ - * / % ( )`, and bit operations; `set /a X+=1` increments. Division truncates. Numbers with a leading `0` are octal, `0x` is hex.
- To delete a variable: `set "NAME="`.
- **Substring and replace:**

```
set "S=Hello World"
echo %S:~0,5%
echo %S:~-5%
echo %S:World=There%
```

Output: `Hello`, `World`, `Hello There`.

14.4 Environment variables and dynamic variables

Some variables are real environment variables (`PATH`, `TEMP`, `USERNAME`). Others are **dynamic**: CMD computes them on each read: `%CD%`, `%DATE%`, `%TIME%`, `%RANDOM%` (0-32767), `%ERRORLEVEL%`, `%CMDCMDLINE%`, `%CMDEXTVERSION%`, `%HIGHESTNUMANODENUMBER%`. They do not show up in a plain `set` listing. Part XXVII has the full table.

14.5 Command-line arguments

Run `demo.cmd one "two words" three`:

Reference	Value
`%0`	`demo.cmd` (how it was invoked)
`%1`	`one`
`%2`	`"two words"` (quotes included)
`%~2`	`two words` (quotes removed)
`%3`	`three`
`%*`	everything after the script name, as typed

`shift` discards `%1` and slides the rest down (`%2` becomes `%1`). Only `%1`-`%9` are directly addressable; `shift` is how you walk more.

```
@echo off
:next
if "%~1"=="" goto :done
echo arg: %~1
shift
goto :next
:done
```

14.6 Validating input

Never assume arguments exist or are sane.

```
if "%~1"=="" (
    echo Usage: %~nx0 ^<folder^>
    exit /b 2
)
if not exist "%~1\" (
    echo Folder not found: "%~1"
    exit /b 3
)
```

("`%~1\"` with a trailing backslash tests specifically for a folder. The carets in the usage line escape < and >.)

Chapter 15. Control Flow: IF, GOTO, FOR and FOR /F

15.1 IF

```
if exist "C:\Windows\notepad.exe" echo found
if not exist "D:\x" echo missing
if defined TEMP echo TEMP is set
if /i "%CHOICE%"=="y" echo yes
if "%A%"=="%B%" echo same text
if %N% gtr 5 echo greater than 5
if errorlevel 1 echo the last exit code was 1 or higher
if %ERRORLEVEL% equ 0 echo exactly zero
```

Comparison operators: `equ neq lss leq gtr geq`. Without them == compares as text. `if /i` ignores case.

ELSE must begin on the same line as the closing parenthesis of the IF block:

```
if exist "%F%" (
    echo present
) else (
    echo absent
)
```

Quoting both sides of == protects against empty variables and spaces. `if %X%==5` becomes `if ==5` when X is empty (syntax error); `if "%X%"=="5"` does not.

`if errorlevel N` is true when the code is **N or greater**, so test the highest value first, or use `%ERRORLEVEL% equ N`.

15.2 GOTO and labels

A **label** is a line starting with a colon, `:name`. `goto :name` jumps there. `goto :eof` ends the script or subroutine (an implicit label at end-of-file). Structured scripts use labels as a table of contents, not as a spaghetti of jumps.

15.3 FOR (files and sets)

```
for %%f in (*.txt) do echo %%f
for %%n in (alpha beta gamma) do echo %%n
for /l %%i in (1,1,5) do echo %%i
for /d %%d in (C:\Windows\*) do echo %%d
for /r C:\Logs %%f in (*.log) do echo %%f
```

Form	Iterates over
`for %%v in (set)`	files or literal words
`for /l %%v in (start,step,end)`	numbers
`for /d %%v in (set)`	directories
`for /r [root] %%v in (set)`	files in a whole tree
`for /f ...`	text lines, strings, or command output

At the prompt use one `%`; in scripts use `%%`.

15.4 FOR /F: parsing text

```
for /f "options" %%v in (file)        do command
for /f "options" %%v in ("string")    do command
for /f "options" %%v in ('command')   do command
```

Option	Meaning	Default
`delims=xyz`	characters that split a line into tokens	space and tab
`tokens=1,3` or `2*`	which tokens to capture; extra tokens go to the next letters; `*` = rest of line	`1`
`skip=n`	lines to skip at the top	0
`eol=c`	lines starting with this character are comments	`;`
`usebackq`	back-quoted text runs a command, double-quoted text names a file, single-quoted text is a literal string	off

Important behaviors:

- **Blank lines are skipped.**
- Each token is assigned to consecutive FOR variable letters: `tokens=1,2 ... %%a %%b`.
- The default `eol=;` means lines starting with `;` are silently dropped; if you must keep them, choose an `eol` character that never begins a line in your data.
- `delims=` with nothing after it means **no delimiters**: the whole line is token 1.
- A command in single quotes is run via `cmd /c`, so `|` inside must be escaped as `^|`.

Examples:

```
rem Who am I, without the domain part
for /f "tokens=2 delims=\" %%u in ('whoami') do echo user=%%u

rem Skip a two-line header, split on commas
for /f "skip=2 tokens=1,3 delims=," %%a in (data.csv) do echo %%a / %%b

rem Whole line, including spaces
for /f "delims=" %%l in (notes.txt) do echo [%%l]

rem File path with spaces: usebackq
for /f "usebackq delims=" %%l in ("C:\My Files\notes.txt") do echo %%l
```

> **NOTE.** On domain-joined machines `whoami` returns `DOMAIN\user`; on standalone PCs it is `COMPUTERNAME\user`. Both parse the same way.

15.5 Nested loops and counters

```
@echo off
setlocal EnableDelayedExpansion
set /a TOTAL=0
for /l %%i in (1,1,3) do (
    for /l %%j in (1,1,2) do (
        set /a TOTAL+=1
        echo i=%%i j=%%j total=!TOTAL!
    )
)
echo Final: %TOTAL%
```

Output ends with `Final: 6`. (Inside the loops `!TOTAL!` is live; after the loops `%TOTAL%` is fine.)

15.6 Arrays and pseudo-data structures

CMD has no arrays. Convention: variables whose names share a prefix.

```
@echo off
setlocal EnableDelayedExpansion
set /a N=0
for %%c in (red green blue) do (
    set /a N+=1
    set "COLOR[!N!]=%%c"
)
for /l %%i in (1,1,%N%) do echo !COLOR[%%i]!
rem list them all
set COLOR[
```

`set COLOR[` prints every variable starting with `COLOR[`. A lookup table keyed by text works the same way: `set "PORT[http]=80"` then `echo !PORT[http]!`.

Chapter 16. Subroutines, Errors, Logging and Testing

16.1 Subroutines with CALL

```
@echo off
setlocal EnableExtensions
call :greet Alice
call :greet "Bob Smith"
call :add 2 3 SUM
echo 2 + 3 = %SUM%
exit /b 0

:greet
echo Hello, %~1
exit /b 0

:add
set /a "%3=%1+%2"
exit /b 0
```

Rules:

- The main code must end with `exit /b` (or `goto :eof`) **before** the first subroutine label, or it will "fall through" into it.
- Arguments arrive as `%1`, `%2`... exactly like script arguments.
- A subroutine returns a **status** with `exit /b N`; a **value** by setting a variable whose name was passed in (`%3` above).
- Use `setlocal`/`endlocal` inside a subroutine to keep its temporaries private. To return a value across `endlocal`:

```
:upper
setlocal
set "R=%~1"
endlocal & set "%~2=%R%"
exit /b 0
```

(The expansion of `%R%` happens before `endlocal` runs: one line, parsed once.)

16.2 ERRORLEVEL and exit codes

Every program returns a number. Patterns:

```
robocopy src dst /e >nul
if errorlevel 8 (echo copy FAILED & exit /b 1)
xcopy a b >nul || (echo xcopy failed & exit /b 1)
```

Key points:

1. `%ERRORLEVEL%` reflects the **most recent** command; capture it at once: `set "RC=%ERRORLEVEL%"`.
2. Never `set ERRORLEVEL=...` yourself. A variable of that name shadows the dynamic value.
3. Many tools use nonzero for non-error states (`find`: 1 = no match; `robocopy`: 1 = files copied).

4. Scripts should exit with meaningful codes so schedulers and CI can react: `exit /b 0` success, `exit /b 2` bad usage, `exit /b 3` missing prerequisite, and so on.

16.3 A reusable logging function

```
:log
rem usage: call :log LEVEL "message"
setlocal
set "T=%TIME: =0%"
>>"%LOGFILE%" echo %DATE% %T:~0,8% [%~1] %~2
echo [%~1] %~2
endlocal
exit /b 0
```

Set `LOGFILE` once at the top (`set "LOGFILE=%~dp0run.log"`). `%TIME: =0%` turns the leading space of an early-morning hour (`9:05:01.12`) into a zero so timestamps sort correctly. `%DATE%` depends on regional settings; for a locale-proof timestamp call PowerShell once: `for /f %%t in ('powershell -NoProfile -Command "Get-Date -Format yyyyMMdd-HHmmss"') do set "STAMP=%%t"`.

SECURITY. Never log passwords, tokens or full command lines that contain them. Logs are copied, emailed and read by many people (Part XIX).

16.4 Privilege and prerequisite checks

```
:require_admin
fltmc >nul 2>&1 || (
    echo This step needs an elevated prompt.
    exit /b 5
)
exit /b 0

:require_tool
where /q %~1 || (
    echo Missing required tool: %~1
    exit /b 3
)
exit /b 0
```

`fltmc` succeeds only for an elevated process, which makes it a convenient test; `net session >nul 2>&1` is another widely used check but depends on the Server service. Neither is a security boundary. Never use them to decide *whether an action is allowed*, only to give clear error messages.

16.5 Temporary files and locking

```
set "TMPF=%TEMP%\myscript_%RANDOM%_%RANDOM%.tmp"
...
del "%TMPF%" 2>nul
```

`%RANDOM%` alone is guessable and collides; combine it with a timestamp and never place temp files in a world-writable location that a less-privileged user can pre-create or replace (Part XIX).

A simple atomic lock uses the fact that `mkdir` fails if the folder exists:

```
mkdir "%TEMP%\myscript.lock" 2>nul || (
    echo Another instance is running.
    exit /b 9
)
rem ... work ...
rmdir "%TEMP%\myscript.lock"
```

If the script can crash, a stale lock remains; real tools store a PID or timestamp and verify it.

16.6 Retry and timeout

```
set /a TRIES=0
:retry
ping -n 2 %TARGET% >nul && goto :ok
set /a TRIES+=1
if %TRIES% lss 3 (
    timeout /t 5 /nobreak >nul
    goto :retry
)
echo Gave up after %TRIES% tries.
exit /b 1
:ok
echo reachable
```

`timeout` fails if stdin is redirected; `ping -n 6 127.0.0.1 >nul` is a five-second-ish portable delay (`-n N` sends N echoes one second apart, so N-1 seconds).

16.7 Debugging

Technique	How
See every command	remove `@echo off`, or `echo on` around the suspect code
Watch values	`echo DEBUG X=[%X%] Y=[!Y!]` with brackets to reveal spaces
Trace a run	`cmd /c script.cmd > trace.txt 2>&1`
Isolate	copy the failing lines to a new file; remove everything else
Pause at a point	`pause` or `set /p DUMMY=Press Enter`
Check line endings and encoding	Chapter 13
Verify quoting	replace the action with `echo` so you see what would run

16.8 Testing batch scripts

A minimal harness: each test calls the unit and compares to an expected value.

```
@echo off
setlocal EnableExtensions
set /a PASS=0, FAIL=0
call :expect "5" "%~dp0add.cmd 2 3"
call :expect "9" "%~dp0add.cmd 4 5"
echo Passed: %PASS%  Failed: %FAIL%
exit /b %FAIL%

:expect
set "GOT="
for /f "delims=" %%r in ('%~2') do set "GOT=%%r"
if "%GOT%"=="%~1" (set /a PASS+=1) else (echo FAIL: %~2 expected %~1 got %GOT% & set /a FAIL+=1)
exit /b 0
```

The idea (not a complete framework; the full framework is a Volume 4 project): the unit prints its result, the harness captures it, compares, counts.

16.9 Performance and portability

- Spawning processes is expensive: prefer built-ins (`findstr` over `type | find` chains; `for /f` over repeated calls).
- Avoid `call` in tight loops (it re-parses); use `goto` loops or inline logic where possible.
- `%DATE%`/`%TIME%` formats, `dir` output, `tasklist` column headers and sort order are **locale-dependent**; parse with `/fo csv /nh` options (when offered) rather than text positions.
- Commands removed or deprecated across versions (`wmic` is deprecated and is disabled by default on Windows 11 24H2 and later; verify on your build and do not depend on it in new scripts). Always check availability with `where /q`.

16.10 When batch is the wrong tool

Need	Better choice
Structured data (JSON, XML, CSV with quoting)	PowerShell, Python
Unicode text	PowerShell, Python
Calls to REST APIs, TLS, credentials	PowerShell, Python, Go
Numbers larger than 32 bits or floating point	PowerShell, Python
Reliable string handling of untrusted input	anything but batch
Long-running services, concurrency	Go, C#, Python

Batch is still the right tool for small bootstrap scripts, recovery media, and gluing existing command-line tools together.

Lab 56. Hello script with header and arguments

Objective. Write a script with a header that validates an argument. **Difficulty.** [B]. **Prerequisites.** Chapter 14. **Environment.** Scratch folder.

Commands. Save as `greet.cmd`:

```
@echo off
setlocal EnableExtensions
if "%~1"=="" (
    echo Usage: %~nx0 name
    exit /b 2
)
echo Hello, %~1! Today is %DATE%.
exit /b 0
```

Run `greet`, then `greet Alice`, then `greet "Mary Ann"`, each followed by `echo %ERRORLEVEL%`.

Expected result. Usage and code `2`; greeting and code `0`; greeting with the full two-word name. **Explanation.** `%~1` strips quotes; the IF tests for an empty first argument. **Verification.** Exit codes as listed. **Troubleshooting.** If `%DATE%` looks odd that is your regional format. **Cleanup.** `del greet.cmd`. **Challenge.** Accept an optional second argument for a greeting word. **Solution.** `set "G=%~2" & if not defined G set "G=Hello"` then `echo %G%, %~1!`.

Lab 57. Argument walker with SHIFT

Objective. Process any number of arguments. **Difficulty.** [B]. **Prerequisites.** Section 14.5. **Environment.** Scratch folder.

Commands. Save as `args.cmd` using the loop from Section 14.5, then run `args one "two words" three`.

Expected result. Three lines: `arg: one`, `arg: two words`, `arg: three`. **Explanation.** Each `shift` moves `%2` into `%1`. **Verification.** Run with no arguments: prints nothing. **Troubleshooting.** An infinite loop means you forgot `shift`. **Cleanup.** `del args.cmd`. **Challenge.** Count the arguments. **Solution.** Add `set /a N=0` before the loop and `set /a N+=1` inside; print `%N%` at `:done`.

Lab 58. IF forms and ELSE placement

Objective. Practice every IF form. **Difficulty.** [B]. **Prerequisites.** Section 15.1. **Environment.** Scratch folder.

Commands. Save as `l58.cmd`:

```
@echo off
set "F=%~1"
if not defined F set "F=%WINDIR%\notepad.exe"
if exist "%F%" (
    echo exists: %F%
) else (
    echo missing: %F%
)
set /a N=7
if %N% gtr 5 echo N is greater than 5
if /i "ABC"=="abc" echo case-insensitive match
find "zzz" "%F%" >nul 2>&1
if errorlevel 1 (echo no match) else (echo match)
```

Run `l58` and `l58 C:\nosuch.txt`.

Expected result. First run: `exists:` line, `N is greater than 5`, `case-insensitive match`, then `no match`. Second run prints `missing:`. **Explanation.** `find` returns 1 when nothing matches. **Verification.** Move `else` to its own line and observe the syntax error. **Troubleshooting.** `) else (` must stay on one line. **Cleanup.**

`del l58.cmd`. **Challenge.** Use `%ERRORLEVEL% equ 1` instead of `if errorlevel 1`. **Solution.** `if %ERRORLEVEL% equ 1 (...) else (...)`.

Lab 59. FOR and FOR /L basics

Objective. Loop over words, files and numbers. **Difficulty.** [B]. **Prerequisites.** Section 15.3. **Environment.** Scratch folder.

Commands.

```
for %i in (alpha beta gamma) do @echo %i
for /l %i in (1,2,9) do @echo %i
> one.txt echo 1
> two.txt echo 2
for %i in (*.txt) do @echo file: %i
for /d %i in (%WINDIR%\*) do @echo dir: %i
```

Expected result. Three words; `1 3 5 7 9`; two file lines (plus any other `.txt` in the folder); a list of folders under Windows (`dir:` prefix). **Explanation.** Single `%` at the prompt. **Verification.** Count with `(for /l %i in (1,1,100) do @echo %i) | find /c /v ""` (prints 100). **Troubleshooting.** In a script use `%%i`. **Cleanup.** `del one.txt two.txt`. **Challenge.** Print only `.txt` files larger than 0 bytes. **Solution.** `for %i in (*.txt) do @if %~zi gtr 0 echo %i`.

Lab 60. FOR /F parsing workshop

Objective. Use delims, tokens, skip and usebackq. **Difficulty.** [I]. **Prerequisites.** Section 15.4. **Environment.** Scratch folder.

Commands. Create the data file, then parse it:

```
> data.csv echo name,city,age
>> data.csv echo Alice,Paris,30
>> data.csv echo Bob,Oslo,25
>> data.csv echo Carol,Rome,41
for /f "skip=1 tokens=1-3 delims=," %a in (data.csv) do @echo %a lives in %b and is %c
for /f "usebackq skip=1 tokens=3 delims=," %a in ("data.csv") do @echo age=%a
for /f "tokens=2 delims=\" %u in ('whoami') do @echo %u
```

Expected result. Three sentences (`Alice lives in Paris and is 30`, ...); three lines `age=30`, `age=25`, `age=41`; your user name. **Explanation.** `skip=1` drops the header; `tokens=1-3` fills `%a %b %c`. **Verification.** Remove `skip=1` and see the header line handled as data. **Troubleshooting.** If tokens are blank, check `delims=`. **Cleanup.** `del data.csv`. **Challenge.** Sum the ages. **Solution.** In a script: `set /a T=0` then `for /f "skip=1 tokens=3 delims=," %%a in (data.csv) do set /a T+=%%a` then `echo %T%` (prints 96).

Lab 61. Delayed expansion counter and accumulator

Objective. Build a string and a counter inside loops. **Difficulty.** [I]. **Prerequisites.** Lab 55, Section 15.5. **Environment.** Scratch folder.

Commands. Save as `l61.cmd`:

```
@echo off
setlocal EnableDelayedExpansion
set "LIST="
set /a N=0
for %%f in (%WINDIR%\*.exe) do (
    set /a N+=1
    if !N! leq 5 set "LIST=!LIST! %%~nxf"
)
echo first five:!LIST!
echo total exe files: %N%
```

Expected result. A list of five `.exe` names and a count. **Explanation.** The IF inside the block must use `!N!`; `%N%` at the end is fine. **Verification.** Change `!N!` to `%N%` in the IF; the list becomes empty or wrong. **Troubleshooting.** If the list is empty confirm `EnableDelayedExpansion`. **Cleanup.** `del l61.cmd`. **Challenge.** Join with commas. **Solution.** Keep a flag and add the comma only after the first item.

Lab 62. Subroutines and returning values

Objective. Write `:add` and `:upper` style functions. **Difficulty.** [I]. **Prerequisites.** Section 16.1. **Environment.** Scratch folder.

Commands. Save the subroutine example from Section 16.1 as `l62.cmd` and run it.

Expected result. `Hello, Alice`, `Hello, Bob Smith`, `2 + 3 = 5`. **Explanation.** `call :label args` enters the subroutine; `exit /b` returns. **Verification.** Remove the first `exit /b 0` (before `:greet`) and watch the code fall through into the subroutines. **Troubleshooting.** `The system cannot find the batch label specified` means a typo in the label. **Cleanup.** `del l62.cmd`. **Challenge.** Add `:max a b` returning the larger value. **Solution.** `if %1 gtr %2 (set "%3=%1") else (set "%3=%2")`.

Lab 63. Logging framework

Objective. Add a timestamped log function. **Difficulty.** [I]. **Prerequisites.** Section 16.3. **Environment.** Scratch folder.

Commands. Save `l63.cmd` with `set "LOGFILE=%~dp0l63.log"`, `call :log INFO "starting"`, `call :log WARN "disk is almost full"`, `call :log INFO "done"`, `exit /b 0` followed by the `:log` function from Section 16.3. Run it twice and `type l63.log`.

Expected result. Six timestamped lines (three per run), appended. **Explanation.** `>>` appends; `%TIME: =0%` pads the hour. **Verification.** Line count: `find /c /v "" l63.log` prints 6. **Troubleshooting.** If the log is not created check write permission on the folder. **Cleanup.** `del l63.cmd l63.log`. **Challenge.** Add a rotation: rename the log to `.old` when it exceeds 100 KB. **Solution.** `for %%s in ("%LOGFILE%") do if %%~zs gtr`

```
102400 move /y "%LOGFILE%" "%LOGFILE%.old" >nul.
```

Lab 64. Retry loop and exit codes

Objective. Retry a flaky check and return a code. **Difficulty.** [I]. **Prerequisites.** Sections 16.2, 16.6. **Environment.** Scratch folder.

Commands. Save the retry snippet from Section 16.6 as `l64.cmd` with `set "TARGET=127.0.0.1"` at the top and `exit /b 0` after `echo reachable`. Run it; change TARGET to `no-such-host.invalid` and run again; after each run `echo %ERRORLEVEL%`.

Expected result. `reachable` with code 0; then three attempts about five seconds apart, `Gave up after 3 tries.` and code 1. **Explanation.** `&&` jumps on success; the counter controls retries. **Verification.** Time the second run (roughly 10-15 seconds). **Troubleshooting.** `timeout` needs an interactive console; replace it with the `ping -n` delay otherwise. **Cleanup.** `del l64.cmd`. **Challenge.** Make the retry count a parameter. **Solution.** `set "MAX=%~1" & if not defined MAX set "MAX=3"` and compare `%TRIES% lss %MAX%`.

Lab 65. Mini test harness

Objective. Test a tiny script automatically. **Difficulty.** [A]. **Prerequisites.** Section 16.8. **Environment.** Scratch folder.

Commands. Create `add.cmd`:

```
@echo off
set /a R=%1+%2
echo %R%
```

then save the harness from Section 16.8 as `test.cmd` in the same folder and run `test`.

Expected result. `Passed: 2 Failed: 0` and exit code 0. **Explanation.** The harness runs each command in a child, captures the last output line and compares it. **Verification.** Change an expected value to `6` and see a `FAIL:` line and exit code 1. **Troubleshooting.** Paths with spaces need extra quoting around `%~2`; keep the lab folder simple. **Cleanup.** `del add.cmd test.cmd`. **Challenge.** Add a test for a missing argument. **Solution.** Make `add.cmd` validate input and expect its error text.

Part XXIV - Hands-On Labs (Beginner Set)

Labs 1-25. Labs 41-55 sit in Parts III-IV and Labs 56-65 in Part V, next to the chapters that teach them. Labs 26-40 and 66 onward are in later volumes.

Chapter 17. Beginner Labs 1-25

> **SAFETY.** Every lab assumes a scratch folder: `mkdir "%TEMP%\cmdlab" & cd /d "%TEMP%\cmdlab"`. Labs marked **VM** should be run in a disposable virtual machine. Cleanup for the whole set: `cd /d "%TEMP%" & rmdir /s /q cmdlab`.

Lab 1. Two kinds of prompt

Objective. Tell a standard prompt from an elevated one. **Difficulty.** [B]. **Prerequisites.** None. **Environment.** Windows 10/11, **VM**.

Commands. Open a normal Command Prompt, then a second one via right-click -> Run as administrator.

```
whoami
whoami /groups | find "Mandatory Label"
title Lab1
```

Expected result. The same user name in both; `Medium Mandatory Level` in the standard prompt and `High Mandatory Level` in the elevated one; the elevated title bar begins with `Administrator:`. **Explanation.** UAC gives admin users two tokens (filtered and full); the integrity label shows which one the process holds. **Verification.** Try `mkdir %WINDIR%\labtest` in both: denied in the standard prompt. **Troubleshooting.** If both show High, your UAC is set to never notify or you are the built-in Administrator; note that and continue. **Cleanup.** Close the elevated window; `rmdir %WINDIR%\labtest` if the directory was created. **Challenge.** Find the integrity level of a third prompt started with `runas /trustlevel:0x20000 cmd` (limited rights). **Solution.** It reports a lower label than Medium in some configurations; record what you see.

Lab 2. Reading the prompt

Objective. Identify every part of the prompt. **Difficulty.** [B]. **Prerequisites.** Chapter 2. **Environment.** Any prompt.

Commands.

```
cd
cd /d %WINDIR%
cd
C:
D:
prompt $P$_$G$S
```

Expected result. `cd` prints the current folder; after `cd /d %WINDIR%` it prints `C:\Windows`; `D:` either switches to D (if present) or says `The system cannot find the drive specified.` The new prompt is two lines. **Explanation.** The prompt shows drive, path and terminator; each drive remembers its own folder. **Verification.** `echo %CD%` matches the `cd` output. **Troubleshooting.** No D: drive is not an error in this lab. **Cleanup.** `prompt $P$G`. **Challenge.** Put the time in the prompt. **Solution.** `prompt $T$S$P$G`.

Lab 3. dir switches

Objective. Use `dir` effectively. **Difficulty.** [B]. **Prerequisites.** Section 3.1. **Environment.** Scratch folder.

Commands.

```
> a.txt echo 1
> bb.txt echo 22
> ccc.log echo 333
dir
dir /b
dir /o:-s
dir /a:-d /b *.txt
dir /s /b
```

Expected result. Listings in the orders described: plain, bare, largest first, only `.txt`, recursive bare. **Explanation.** Switch letters are combinable; `/o:-s` sorts descending by size. **Verification.** The sizes are 3, 4, 5 bytes (digits plus CRLF). **Troubleshooting.** A colon is required in `/a:` and `/o:` forms on some builds; include it. **Cleanup.** `del a.txt bb.txt ccc.log`. **Challenge.** List only the newest file. **Solution.** `dir /b /o:-d` and take the first line, or `for /f %f in ('dir /b /o:-d') do @echo %f & goto :eof` in a script.

Lab 4. Navigation drill

Objective. Move around the tree confidently. **Difficulty.** [B]. **Prerequisites.** Section 3.2. **Environment.** Scratch folder.

Commands.

```
md a\b\c
cd a\b\c
cd ..
cd ..\..
cd
cd \
cd
cd /d "%TEMP%\cmdlab"
```

Expected result. You descend three levels, climb back, reach the drive root, then return to the lab folder. **Explanation.** `..` is the parent; `\` is the root of the current drive. **Verification.** `cd` after each step. **Troubleshooting.** A failed `cd` leaves you where you were; read the message. **Cleanup.** `rd /s /q a`. **Challenge.** Reach the lab folder from anywhere in one command. **Solution.** `cd /d "%TEMP%\cmdlab"`.

Lab 5. Making and removing folders

Objective. Create a tree and remove it safely. **Difficulty.** [B]. **Prerequisites.** Lab 4. **Environment.** Scratch folder.

Commands.

```
md project\src project\docs project\tmp
tree /a project
rd project\tmp
rd project
rd /s /q project
dir /b
```

Expected result. The tree appears; `rd project\tmp` succeeds (empty); `rd project` fails with `The directory is not empty.`; `rd /s /q project` removes everything. **Explanation.** `rd` refuses non-empty folders unless `/s`. **Verification.** `dir /b` no longer lists `project`. **Troubleshooting.** `The process cannot access the file...` means another program holds a file open in it. **Cleanup.** Nothing left. **Challenge.** Write a safe one-liner that removes a folder only if it exists. **Solution.** `if exist "project\" rd /s /q "project"`.

Lab 6. Creating files with redirection

Objective. Create, overwrite and append. **Difficulty.** [B]. **Prerequisites.** Section 3.3. **Environment.** Scratch folder.

Commands.

```
echo first> f.txt
type f.txt
echo second> f.txt
type f.txt
echo third>> f.txt
type f.txt
```

Expected result. `first`; then `second` (overwritten); then `second` and `third`. **Explanation.** `>` truncates, `>>` appends. **Verification.** `find /c /v "" f.txt` prints 2. **Troubleshooting.** A space before `>` adds a trailing space in the file. **Cleanup.** `del f.txt`. **Challenge.** Create a file with an empty first line. **Solution.** `echo.> f.txt` then `echo text>> f.txt`.

Lab 7. Viewing files

Objective. Use `type` and `more` on long output. **Difficulty.** [B]. **Prerequisites.** Lab 6. **Environment.** Scratch folder.

Commands.

```
dir /b %WINDIR%\System32 > list.txt
type list.txt | find /c /v ""
more list.txt
more +100 list.txt
```

Expected result. A large line count; `more` shows a page at a time (Space next, Q quit); `+100` starts at line 100. **Explanation.** `more` is a pager reading a file or stdin. **Verification.** Compare counts with `dir /b %WINDIR%\System32 | find /c /v ""`. **Troubleshooting.** Press Q if stuck. **Cleanup.** `del list.txt`. **Challenge.** Show only lines containing `ping`. **Solution.** `findstr /i ping list.txt`.

Lab 8. Copy, rename, move

Objective. Duplicate and reorganize files. **Difficulty.** [B]. **Prerequisites.** Lab 6. **Environment.** Scratch folder.

Commands.

```
echo data> orig.txt
copy orig.txt copy1.txt
ren copy1.txt copy2.txt
md archive
move copy2.txt archive\
dir /b /s
```

Expected result. Two files remain: `orig.txt` and `archive\copy2.txt`. **Explanation.** `ren` renames in place; `move` relocates. **Verification.** `fc orig.txt archive\copy2.txt` reports no differences. **Troubleshooting.** `ren` cannot take a path in the new name. **Cleanup.** `rd /s /q archive & del orig.txt`. **Challenge.** Copy without an overwrite prompt when the target exists. **Solution.** `copy /y orig.txt archive\copy2.txt`.

Lab 9. Deleting safely

Objective. Build the habit of previewing before deleting. **Difficulty.** [B]. **Prerequisites.** Lab 8. **Environment.** Scratch folder.

Commands.

```
> x1.tmp echo 1
> x2.tmp echo 2
> keep.txt echo k
dir *.tmp
del /p *.tmp
del *.tmp
dir /b
```

Expected result. `del /p` asks per file (answer `N` to keep, then run the plain `del`); finally only `keep.txt` remains. **Explanation.** Previewing with `dir` using the same pattern shows exactly what `del` will hit. **Verification.** `dir /b` output. **Troubleshooting.** `Could Not Find` means the pattern matched nothing. **Cleanup.** `del keep.txt`. **Challenge.** Delete only files older than today. **Solution.** `forfiles /d -1 /m *.tmp /c "cmd /c del @file"` (forfiles is covered in Volume 3).

Lab 10. Tree and structure documentation

Objective. Produce a folder map. **Difficulty.** [B]. **Prerequisites.** Lab 5. **Environment.** Scratch folder.

Commands.

```
md site\css site\js
> site\index.html echo x
> site\css\s.css echo x
tree /f /a site > map.txt
type map.txt
```

Expected result. An ASCII tree including files. **Explanation.** `/a` avoids box-drawing characters that can break in other encodings. **Verification.** `find /c /v "" map.txt`. **Troubleshooting.** Garbled lines mean you omitted `/a`. **Cleanup.** `rd /s /q site & del map.txt`. **Challenge.** List only folder names. **Solution.** `dir /s /b /a:d site`.

Lab 11. Help system tour

Objective. Find syntax yourself. **Difficulty.** [B]. **Prerequisites.** Section 3.3. **Environment.** Any prompt.

Commands.

```
help
help copy
xcopy /?
robocopy /? | more
where /?
```

Expected result. Command lists and syntax screens. **Explanation.** `help` covers built-ins; `/?` covers external tools. **Verification.** Try `help ping` and read the message. **Troubleshooting.** Long output: pipe to `more`. **Cleanup.** None. **Challenge.** Save the `for` help to a file and count the lines. **Solution.** `help for > forhelp.txt & find /c /v "" forhelp.txt & del forhelp.txt`.

Lab 12. Customizing the console

Objective. Change title, colors and prompt, then reset. **Difficulty.** [B]. **Prerequisites.** Chapter 3. **Environment.** Any prompt.

Commands.

```
title Practice
color 0A
prompt $T$G
cls
color
prompt $P$G
```

Expected result. Green-on-black text and a time prompt, then everything reset. **Explanation.** `color` with no argument restores defaults. **Verification.** The title bar shows `Practice`. **Troubleshooting.** Some hosts ignore `color` for the whole window (Windows Terminal applies it to the text only). **Cleanup.** Close the window. **Challenge.** Create a red-title production warning. **Solution.** `title PRODUCTION - BE CAREFUL & color 4F`.

Lab 13. Environment variables

Objective. Read, create and delete variables. **Difficulty.** [B]. **Prerequisites.** Section 2.6. **Environment.** Any prompt.

Commands.

```
echo %USERNAME% on %COMPUTERNAME%
set LAB=hello
echo %LAB%
set LAB
set "LAB="
set LAB
echo %RANDOM% %RANDOM%
```

Expected result. Your names; `hello`; the listing `LAB=hello`; `Environment variable LAB not defined`; two random numbers. **Explanation.** Variables live only in this process and its children. **Verification.** Open a second prompt: `echo %LAB%` shows the literal text `%LAB%`. **Troubleshooting.** A space around `=` creates a variable with a space in its name. **Cleanup.** None. **Challenge.** Print just the first three characters of your user name. **Solution.** `echo %USERNAME:~0,3%`.

Lab 14. Arithmetic with set /a

Objective. Compute with integers. **Difficulty.** [B]. **Prerequisites.** Lab 13. **Environment.** Any prompt.

Commands.

```
set /a A=7*6
echo %A%
set /a B=A+8
set /a C=10/3
set /a D=10%3
echo %B% %C% %D%
set /a E=2147483647+1
echo %E%
```

Expected result. `42`, then `50 3 1`, then a wrapped negative number (`-2147483648`). **Explanation.** Values are 32-bit signed integers; division truncates. **Verification.** `set /a X=08` gives an error (octal). **Troubleshooting.** At the prompt use one `%` for modulo; inside a script write `%%`. **Cleanup.** `set A=&set B=&set C=&set D=&set E=`. **Challenge.** Average of 7, 9, 12 as an integer. **Solution.** `set /a AVG=(7+9+12)/3` -> 9.

Lab 15. ERRORLEVEL hunt

Objective. Map commands to exit codes. **Difficulty.** [B]. **Prerequisites.** Section 2.5. **Environment.** Scratch folder.

Commands.

```
echo x> t.txt
find "x" t.txt >nul
echo find-hit=%ERRORLEVEL%
find "zzz" t.txt >nul
echo find-miss=%ERRORLEVEL%
find "x" nosuch.txt >nul 2>&1
echo find-nofile=%ERRORLEVEL%
cd nosuchdir 2>nul
echo cd=%ERRORLEVEL%
cmd /c exit 42
echo custom=%ERRORLEVEL%
```

Expected result. `0`, `1`, `2`, `1`, `42`. **Explanation.** Nonzero does not always mean disaster (`find` miss). **Verification.** Repeat and record. **Troubleshooting.** Each `echo` must be on its own line right after its command. Putting `& echo %ERRORLEVEL%` on the same line shows a stale value, because `%ERRORLEVEL%` is expanded before the line runs (Chapter 10). **Cleanup.** `del t.txt`. **Challenge.** Build a table of five more commands. **Solution.** Try `ping -n 1 127.0.0.1`, `ping -n 1 bad.invalid`, `dir nosuch`, `type nosuch`, `where nosuch`.

Lab 16. stdout versus stderr

Objective. Separate the two streams. **Difficulty.** [B]. **Prerequisites.** Section 2.4. **Environment.** Scratch folder.

Commands.

```
dir C:\Windows nosuchfolder > out.txt 2> err.txt
type out.txt | find /c /v ""
type err.txt
dir C:\Windows nosuchfolder > both.txt 2>&1
type both.txt | find /c /v ""
```

Expected result. `out.txt` has the Windows listing; `err.txt` has the error text; `both.txt` has both. **Explanation.** Two streams, two targets. **Verification.** Compare line counts. **Troubleshooting.** `dir` accepts several paths and lists each. **Cleanup.** `del out.txt err.txt both.txt`. **Challenge.** Show errors on screen but hide normal output. **Solution.** `dir C:\Windows nosuchfolder > nul`.

Lab 17. History and keyboard shortcuts

Objective. Work faster at the prompt. **Difficulty.** [B]. **Prerequisites.** Section 3.6. **Environment.** Interactive prompt.

Commands. Run `echo one`, `echo two`, `echo three`. Then press Up twice, F7, F3, F8, and type `ec` followed by F8. Finally:

```
doskey /history
doskey ll=dir /a $*
ll C:\
```

Expected result. History recall works; the macro `ll` lists C:\ with hidden items. **Explanation.** `doskey` macros expand before the parser runs; `$*` is all arguments. **Verification.** `doskey /macros`. **Troubleshooting.** Macros vanish when the window closes. **Cleanup.** `doskey ll=`. **Challenge.** Create a macro `cdd` for `cd /d`. **Solution.** `doskey cdd=cd /d $*`.

Lab 18. Where does a command come from?

Objective. Use `where`, `path` and `PATHEXT`. **Difficulty.** [B]. **Prerequisites.** Chapter 12. **Environment.** Any prompt.

Commands.

```
where notepad
where /q calc
echo %ERRORLEVEL%
path
echo %PATHEXT%
path %PATH%;C:\nosuch
path
```

Expected result. Paths to `notepad.exe` and exit 0 for `calc`; the PATH list; the modified list (this window only). **Explanation.** PATH is a search list; changes are session-local. **Verification.** Open a second prompt: the PATH is unchanged. **Troubleshooting.** A long PATH wraps awkwardly; use `for` to list it. **Cleanup.** Close the window. **Challenge.** Find the second `find.exe` on a machine with Git installed. **Solution.** `where find` (two lines if Git's is present).

Lab 19. find and findstr

Objective. Filter text. **Difficulty.** [B]. **Prerequisites.** Section 8. **Environment.** Scratch folder.

Commands.

```
> log.txt echo 10:00 INFO start
>> log.txt echo 10:01 ERROR disk full
>> log.txt echo 10:02 info retry
>> log.txt echo 10:03 ERROR timeout
find "ERROR" log.txt
find /i /c "info" log.txt
findstr /n /i "error" log.txt
findstr /v /i "info" log.txt
```

Expected result. Two ERROR lines; count `2` for info (case-insensitive); numbered error lines (2 and 4); lines without info. **Explanation.** `find` is literal; `findstr` supports regex and more switches. **Verification.** `findstr /r "^10:0[23]" log.txt`. **Troubleshooting.** `find` needs quotes around the string. **Cleanup.** `del log.txt`. **Challenge.** Count ERROR lines only. **Solution.** `find /c "ERROR" log.txt`.

Lab 20. Pipes and sort

Objective. Chain commands. **Difficulty.** [B]. **Prerequisites.** Lab 19. **Environment.** Scratch folder.

Commands.

```
dir /b %WINDIR%\System32\*.exe | sort | more
dir /b %WINDIR%\System32\*.exe | find /c /v ""
dir /b %WINDIR%\System32\*.exe | findstr /i "^p" | sort /r
```

Expected result. Sorted names, a count, and names starting with `p` in reverse order. **Explanation.** Each `|` connects one program's output to the next one's input. **Verification.** The `find /c` count matches the `dir` file count line. **Troubleshooting.** Press Q to leave `more`. **Cleanup.** None. **Challenge.** Show the five alphabetically last names. **Solution.** `dir /b %WINDIR%\System32\*.exe | sort /r` and read the first five lines (the `head` tool does not exist in CMD).

Lab 21. Hidden and read-only attributes

Objective. Change attributes and find hidden items. **Difficulty.** [B]. **Prerequisites.** `attrib` card. **Environment.** Scratch folder.

Commands.

```
echo secret> h.txt
attrib +h +r h.txt
dir /b
dir /b /a:h
attrib h.txt
del h.txt
attrib -h -r h.txt
del h.txt
```

Expected result. `h.txt` invisible in plain `dir`, visible with `/a:h`; `attrib` shows `A SH R`-style flags (here `A H R`); the first `del` fails with `Access is denied.` (read-only); after clearing flags it succeeds. **Explanation.** Hidden is cosmetic, not security; read-only blocks plain delete. **Verification.** `dir /b` is empty at the end. **Troubleshooting.** If your first `del` unexpectedly succeeds, check `attrib` to confirm the read-only flag was actually set. **Cleanup.** Already done. **Challenge.** Show all hidden files under the lab folder. **Solution.** `dir /s /b /a:h`.

Lab 22. System identity report

Objective. Gather basic facts into a file. **Difficulty.** [B]. **Prerequisites.** Section 9. **Environment.** Scratch folder.

Commands.

```
(
echo Host: %COMPUTERNAME%
echo User: %USERDOMAIN%\%USERNAME%
ver
echo Date: %DATE%  Time: %TIME%
echo Temp: %TEMP%
) > report.txt
type report.txt
```

Expected result. A five-line report (the `ver` output adds a blank line). **Explanation.** The parenthesized group redirects all its output once. **Verification.** `type report.txt`. **Troubleshooting.** Put a space before `)` only if a line ends in `)`. **Cleanup.** `del report.txt`. **Challenge.** Add the free space on C:. **Solution.** `for /f "tokens=3" %a in ('dir C:\ ^| find "bytes free"') do @echo Free: %a` (locale-dependent; fine for a beginner lab).

Lab 23. First batch file

Objective. Save and run a script. **Difficulty.** [B]. **Prerequisites.** Chapter 14. **Environment.** Scratch folder.

Commands. Create `first.cmd` containing:

```
@echo off
echo Script name: %~nx0
echo Running in : %CD%
echo Arguments  : %*
pause
```

Run `first`, `first a b c`, and double-click it from Explorer.

Expected result. The name, folder and arguments print; `pause` waits. **Explanation.** `%0` is the script, `%*` the arguments. **Verification.** Remove `pause` and double-click: the window flashes and closes. **Troubleshooting.** If Notepad added `.txt`, choose All Files in Save As. **Cleanup.** `del first.cmd`. **Challenge.** Show the script's own folder. **Solution.** `echo Folder: %~dp0`.

Lab 24. Script that makes a report with checks

Objective. Combine IF, FOR and redirection. **Difficulty.** [I]. **Prerequisites.** Lab 23. **Environment.** Scratch folder.

Commands. Save as `inv.cmd`:

```
@echo off
setlocal EnableExtensions
set "OUT=%~dp0inv.txt"
> "%OUT%" echo Inventory of %~1
if "%~1"=="" (echo Usage: %~nx0 folder & exit /b 2)
if not exist "%~1\" (echo No such folder & exit /b 3)
set /a N=0
for %%f in ("%~1\*") do set /a N+=1
>> "%OUT%" echo Files: %N%
>> "%OUT%" dir /b /a:-d "%~1"
type "%OUT%"
```

Run `inv %WINDIR%` then `inv C:\nosuch`.

Expected result. A report listing the file count and names; exit codes 0 and 3. **Explanation.** Validation first, then work, then output. **Verification.** `echo %ERRORLEVEL%` after each run. **Troubleshooting.** Paths with spaces need the quotes exactly as written. **Cleanup.** `del inv.cmd inv.txt`. **Challenge.** Also count folders. **Solution.** Add `for /d %%d in ("%~1\*") do set /a D+=1` and print `D`.

Lab 25. Beginner capstone: safe file organizer

Objective. Sort a folder's files into subfolders by extension, with a dry-run mode. **Difficulty.** [I]. **Prerequisites.** Labs 1-24. **Environment.** Scratch folder, only test data.

Commands. Create test files, then save `organize.cmd`:

```
md inbox
for %i in (a.txt b.txt c.log d.csv e.txt) do @echo x> inbox\%i
```

```
@echo off
setlocal EnableExtensions EnableDelayedExpansion
rem usage: organize.cmd folder [/go]   (default is a dry run)
if "%~1"=="" (echo Usage: %~nx0 folder [/go] & exit /b 2)
if not exist "%~1\" (echo Not a folder: %~1 & exit /b 3)
set "GO=0"
if /i "%~2"=="/go" set "GO=1"
for %%f in ("%~1\*.*") do (
    set "EXT=%%~xf"
    if defined EXT (
        set "EXT=!EXT:~1!"
        if "!GO!"=="1" (
            if not exist "%~1\!EXT!\" md "%~1\!EXT!"
            move /y "%%~f" "%~1\!EXT!\" >nul
            echo moved %%~nxf -> !EXT!\
        ) else (
            echo would move %%~nxf -^> !EXT!\
        )
    )
)
```

Run `organize inbox` (dry run), then `organize inbox /go`, then `tree /f /a inbox`.

Expected result. The dry run prints five `would move` lines and changes nothing; the real run creates `txt`, `log`, `csv` subfolders and moves files; the tree shows them. **Explanation.** A dry-run default is the most important safety habit for any script that changes files. Delayed expansion is needed because `EXT` changes inside the loop. **Verification.** `dir /b inbox` shows only the three new folders. **Troubleshooting.** Files without extensions are skipped by the `if defined EXT` test. **Cleanup.** `rd /s /q inbox & del organize.cmd`. **Challenge.** Add a summary count of moved files. **Solution.** `set /a M=0` at the top, `set /a M+=1` after each `move`, `echo Moved !M! files` at the end.

Back Matter

Glossary, command index, references and what comes next.

Chapter 18. Glossary

Term	Definition
Argument	A piece of text following a command name that tells it what to work on
Batch file	A `.bat` or `.cmd` text file whose lines CMD runs in order
Block	Commands grouped in parentheses; parsed as a single unit
Built-in (internal) command	A command implemented inside `cmd.exe` itself (`cd`, `set`, `echo`)
Caret (^)	CMD's escape character outside double quotes
CMD / cmd.exe	The Windows command interpreter
Code page	A table mapping byte values to characters for console or ANSI text
Console host	The system component (`conhost.exe`) that owns a console window
Current directory	The folder relative paths are resolved against; one per process (and per drive in CMD)
Delayed expansion	Replacing `!VAR!` at execution time rather than at parse time
Dynamic variable	A variable CMD computes on each read (`%CD%`, `%RANDOM%`, `%ERRORLEVEL%`)
Elevated	Running with the full (unfiltered) administrator token after UAC consent
Environment variable	A named text value inherited by child processes
ERRORLEVEL	CMD's name for the exit code of the last command
Exit code	A number a finished program returns; 0 conventionally means success
External command	A separate executable file found by searching (`ping.exe`)
FOR variable	The single-letter loop variable (`%%i` in scripts, `%i` at the prompt)
Handle (redirection)	The number naming a stream: 0 stdin, 1 stdout, 2 stderr
Integrity level	A label (Low, Medium, High, System) attached to a process token
Kernel mode / user mode	Privileged CPU mode for the OS core versus normal mode for applications
PATH	A list of folders CMD searches for external commands
PATHEXT	A list of file extensions CMD appends when you omit one
Pipe (|)	Connects one command's stdout to another's stdin; each side runs in its own CMD
Redirection	Sending a stream to or from a file or device instead of the console
Shell	A program that reads commands and runs other programs
stderr / stdout / stdin	Standard error, output and input streams
Subroutine	A labeled section of a batch file invoked with `call :label`
Token (parsing)	A piece of a line produced by splitting on delimiters (FOR /F)
UAC	User Account Control: the mechanism that separates standard and elevated tokens
Windows Terminal	A modern tabbed terminal application; hosts shells but is not one

Chapter 19. Command Index (Volume 1)

Command	Where the card is
`assoc`, `ftype`, `chcp`, `doskey`	Chapter 9
`attrib`	Chapter 6
`call`, `exit`	Chapter 7
`cd`, `chdir`	Chapter 5
`choice`, `timeout`, `pause`	Chapter 7
`cls`, `title`, `color`, `prompt`	Chapter 5
`copy`, `xcopy`, `move`	Chapter 6
`date`, `time`, `ver`, `vol`, `hostname`	Chapter 9
`del`, `erase`, `ren`, `rename`	Chapter 6
`dir`	Chapter 5
`echo`	Chapter 5
`fc`, `comp`	Chapter 8
`find`, `findstr`	Chapter 8
`help`, `/?`	Chapter 5
`md`, `mkdir`, `rd`, `rmdir`	Chapter 6
`mklink`	Chapter 6
`path`	Chapter 7
`pushd`, `popd`	Chapter 6
`set`	Chapter 7
`sort`, `clip`	Chapter 8
`start`	Chapter 7
`tree`	Chapter 5
`type`, `more`	Chapter 5
`where`	Chapter 6
`for`, `if`, `goto`, `shift`, `setlocal`	Chapters 14-16 (language chapters rather than cards)

Not yet carded (planned for later volumes with their subject chapters): `robocopy`, `icacls`, `takeown`, `fsutil`, `cipher`, `compact`, `tasklist`, `taskkill`, `systeminfo`, `sc`, `net`, `whoami`, `runas`, `reg`, `schtasks`, `wevtutil`, `eventcreate`, networking tools, recovery tools, `certutil`, `bitsadmin`, `regsvr32`, `rundll32`, `mshta`, `wmic`, `forfiles`, `msiexec`, `installutil`, `cscript`, `wscript`.

Chapter 20. References and Verification Notes

Primary sources for this volume (verify each command against the page for your exact Windows build before relying on it):

- Microsoft Learn, *Windows commands* reference (the index of CMD built-ins and utilities): https://learn.microsoft.com/windows-server/administration/windows-commands/windows-commands
- Microsoft Learn, *cmd*, *set*, *for*, *if*, *call*, *start*, *where*, *findstr*, *robocopy* command pages (linked from the index above).
- Microsoft Learn, *User Account Control* and *Windows integrity levels* documentation (for Chapter 1 and Part X material).
- Microsoft Learn, Sysinternals documentation, for Process Explorer, Process Monitor and Sysmon (used in later volumes).
- MITRE ATT&CK, technique T1059.003 *Command and Scripting Interpreter: Windows Command Shell*, for the attacker-side view used in Volume 3.

A note on the parser chapters. Microsoft does not publish the internal grammar of `cmd.exe`. The phase model in Chapter 10 is the community-standard description of observed behavior, and every claim is paired with an experiment. Treat the experiments as the source of truth, and report any difference you observe on your build.

Items to verify on your own machine. These were written from documented behavior and widely reported findings, but details can differ between Windows builds: the exact wording of error messages, the glued-digit redirection case (Lab 52), the `NoDefaultCurrentDirectoryInExePath` result (Lab 47), UTF-8 BOM handling in batch files, and WMIC availability on Windows 11 24H2 and later.

Chapter 21. What Comes Next

Volume 2 (planned): advanced batch engineering (Part VI), filesystem and NTFS (Part VII), processes (VIII), services (IX), users, groups and tokens (X), registry (XI), scheduled tasks (XII), networking (XIII), firewall (XIV), event logging (XV), Defender (XVI), repair and recovery (XVII) and performance (XVIII), with the administration and networking labs and the matching command cards.

Volume 3 (planned): defensive security research, command injection and batch security, path-resolution security, incident response and forensics, all built on benign simulations.

Volume 4 (planned): the remaining labs, the 17 projects, the capstone audit toolkit, the troubleshooting encyclopedia, expert reference tables, CMD versus PowerShell, and the combined glossary and index.